TABLE OF CONTENTS

CHAPTER NO:	TITLE	PAGE NO:
	ABSTRACT	i
	LIST OF TABLES	ii
	LIST OF FIGURES	iii
	LIST OF ABBREVIATIONS	iv
1	**INTRODUCTION**	1
	1.1 OVERVIEW OF THE PROJECT	1
	1.2 LITERATURE SURVEY	2
	1.2.1 AI in Design Automation	3
	1.2.2 Natural Language Processing for Requirement Understanding	3
	1.2.3 Text-to-3D Generation	4
	1.2.4 Natural Language Processing for Requirement Understanding	4
	1.2.5 Procedural 3D Modelling	5
	1.2.6 AI-Assisted Architectural Visualization	5
	1.3 PROPOSED SYSTEM OBJECTIVES AND SCOPES	6
	1.3.1 Objectives of the Proposed system	6
	1.3.2 Scope of the Proposed system	7-9
2	**REQUIREMENTS SPECIFICATIONS**	9
	2.1 OVERALL DESCRIPTION	10-11
	2.1.1 Functional Requirements	11-13
	2.2 PRODUCT PERSPECTIVE	14
	2.3 PRODUCT FUNCTIONS	14
	2.3.1 User Requirement Input	15
	2.3.2 AI-Based Requirement Processing	15
	2.3.3 House Requirement Extraction	15
	2.3.4 Requirement Structuring	15
	2.3.5 Requirement Validation	15

	2.3.6 3D House Generation	16
	2.3.7 Architectural Component Generation	16
	2.3.8 Multi-Floor House Generation	16
	2.3.9 Result Visualization	16
	2.3.10 Error Handling and Feedback	16
	2.4 USER CHARACTERISTICS	16
	2.4.1 General Users	17
	2.4.2 Required Knowledge	17
	2.4.3 User Interaction	17
	2.4.4 Technical Knowledge	17
	2.4.5 Expected User Behavior	17
	2.5 OPERATING ENVIRONMENT CONSTRAINTS	17-18
	2.5.1 Software Specification	18-19
	2.5.2 Hardware Specification	19
	2.6 SPECIFIC REQUIREMENTS	19
	2.6.1 User Requirements	19
	2.6.2 Admin Requirements	20
	2.6.3 Functional Requirements	20
	2.6.4 Non- Functional Requirements	20-21
	2.6.5 Security Requirements	21-22
	2.7 EXTERNAL INTERFACE REQUIREMENTS	22
	2.7.1 User Interface	22
	2.7.2 Frontend–Backend Interface	22
	2.7.3 AI Processing Interface	22
	2.7.4 Backend–Blender Interface	22
	2.7.5 Output Interface	23
	2.7.6 Communication Interface	23
	2.8 SYSTEM FEATURES	23
	2.8.1 Natural Language House Requirement Input	23
	2.8.2 AI-Based Requirement Understanding	23
	2.8.3 Structured House Specification	23
	2.8.4 Requirement Validation	23
	2.8.5 Procedural 3D House Generation	23

	2.8.6 Multi-Floor Support	26
	2.8.7 Frontend and Backend Integration	26
	2.8.8 3D Visualization	26
	2.8.9 Error and Status Feedback	26
3	**SYSTEM DESIGN AND TEST PLAN**	24-25
	3.1 DATA FLOW DIAGRAM (DFD)	25
	3.1.1 Level 0 DFD	25
	3.1.2 Level 1 DFD	26-27
	3.2 USE CASE DIAGRAM	27-28
	3.3 TABLE DESIGNS	29
	3.3.1 HOUSE REQUIREMENT DATA STRUCTURE	29-30
	3.3.2 DATA PROCESSING	30
	3.4 PERFORMANCE REQUIREMENTS	30
	3.4.1 Requirement Processing	31
	3.4.2 Validation Performance	31
	3.4.3 3D Generation Performance	31
	3.4.4 Response Time	31
	3.4.5 Resource Utilization	31
	3.4.6 Scalability	31
	3.4.7 Reliability During Processing	31
	3.5 SOFTWARE QUALITY ATTRIBUTES	32
	3.5.1 Usability	32
	3.5.2 Reliability	32
	3.5.3 Performance	32
	3.5.4 Maintainability	32
	3.5.5 Scalability	32
	3.5.6 Flexibility	32
	3.5.7 Security	33
	3.5.8 Compatibility	33
	3.5.9 Accuracy of Requirement Interpretation	33
	3.6 DETAILED DESIGN	33
	3.6.1 User Interface Module	33
	3.6.2 AI Requirement Parser Module	33

	3.6.3 Structured Requirement Module	33-34
	3.6.4 Validation Module	34
	3.6.5 Backend Processing Module	34-35
	3.6.6 Blender 3D Generation Module	35
	3.6.7 Result Processing Module	35
	3.6.8 Overall Processing Sequence	35-36
	3.7 TEST CASES	36
	3.7.1 Test Case for user	36-37
4	**IMPLEMENTATION AND RESULTS**	38
	4.1 IMPLEMENTATION	38
	4.1.1 Backend Development	38-39
	4.1.2 Frontend Development	39
	4.1.3 Integration and Testing	40-41
	4.2 IMPLEMENTATION RESULTS	41-42
5	**CONCLUSION AND FUTURE WORKS**	43
	5.1 CONCLUSION	43-44
	5.2 FUTURE WORKS	44-46
	5.2.1 Future Implementation Plan	46-47
	REFERENCES	48
	APPENDIXES	50
	Appendix A- Screenshots	50-55
	Appendix B- Sample Code	55-91

ABSTRACT

Designing a house in three-dimensional form traditionally requires architectural knowledge, specialized software, and considerable manual effort. The proposed project, **AI-Based Text-to-3D House Generation System**, provides an intelligent approach for converting natural language house requirements into a structured and interactive 3D house model. The system allows users to describe their desired house using simple text, such as the number of floors, number of bedrooms, room requirements, and other structural constraints.

The system uses an **Artificial Intelligence-based requirement parser** to understand the user's natural language input and convert it into structured house requirements. Important information such as house type, number of bedrooms, number of floors, room requirements, and floor constraints is extracted from the input. The structured requirements are then validated to ensure that the requested constraints are correctly identified and applied.

The processed house requirements are communicated through the backend API to the **Blender-based 3D generation module**. Blender uses the structured information to procedurally create the required architectural elements, including floors, rooms, walls, doors, windows, and other basic components of the house. This enables the system to generate a three-dimensional representation without requiring the user to manually create the complete model.

The system uses modern web technologies including **React** for the frontend interface and backend API technologies for communication between the user interface, AI processing module, and Blender generation module. The architecture is designed to provide a simple workflow where the user enters requirements in natural language and receives a corresponding 3D house model.

The proposed system reduces the complexity of traditional house modelling and demonstrates how artificial intelligence, natural language processing, backend services, and procedural 3D modelling can be integrated into a single application. The project can be further extended with advanced architectural layouts, furniture placement, material generation, realistic rendering, and more detailed customization options.

LIST OF TABLES

TABLE NO:	TITLE	PAGE NO:
2.1	Functional Requirements of the System	11
2.2	User Characteristics	17
2.3	Software Specifications	18
2.4	Hardware Specifications	18-19
2.5	Functional Requirements	20
2.6	Non-Functional Requirements	20-21
2.7	Security Requirements	21-22
3.1	House Constraint Validation Test Cases	26
3.2	3D House Generation Test Cases	27
3.3	3D House Generation Test Cases	28

LIST OF FIGURES

FIGURE NO:	TITLE	PAGE NO:
3.1	DFD Level 0	26
3.2	DFD Level 1	27
3.3	User Use Case Diagram	28
3.4	System Architecture	28
A.1	Home Page	50
A.2	House Requirement Input Page	51
A.3	Parsed House Requirements	52
A.6	Final 3D House Output	52

LIST OF ABBREVIATIONS

LIST	ABBREVIATIONS	MEANING
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	BHK	Bedroom, Hall and Kitchen
4	CSS	Cascading Style Sheets
5	DFD	Data Flow Diagram
6	HTML	Hypertext Markup Language
7	HTTP	Hypertext Transfer Protocol
8	JSON	JavaScript Object Notation
9	NLP	Natural Language Processing
10	REST	Random Forest
11	UI	User Interface
12	UX	User Experience
13	3D	Three-Dimensional
14	VR	Virtual Reality



CHAPTER 1
INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

The **AI-Based Text-to-3D House Generation System** is an intelligent design-assistance system developed to simplify the process of creating conceptual three-dimensional house models from natural-language requirements. Traditional house modelling generally requires architectural knowledge, manual planning, specialized design software, and considerable time to create rooms, walls, floors, doors, windows, and other structural components. These requirements can make the initial design and visualization process difficult for users who do not have advanced 3D modelling skills.

The proposed system provides a user-friendly approach in which users can describe their desired house using simple natural language. For example, a user can provide a requirement such as *"Create a modern 4BHK house with 2 floors, with 2 bedrooms on the ground floor and 2 bedrooms on the first floor."* Instead of manually creating the complete model, the system analyses the given description and identifies important requirements such as the house type, number of bedrooms, number of floors, room requirements, and floor-wise constraints.

The system uses an **AI-assisted requirement parsing module** to convert the user's natural-language description into a structured house specification. Important information such as BHK count, floor count, bedrooms, bathrooms, kitchen, living areas, dining areas, balconies, and other supported requirements can be extracted from the input. For example, the term **4BHK** is interpreted as a requirement for four bedrooms. The extracted information is then organized into a structured data format that can be processed by the remaining modules of the system.

A validation mechanism is included to verify whether the extracted requirements are logically consistent before the 3D generation process begins. This is particularly important for handling constraints related to the number of floors and the distribution of rooms between floors. For instance, when a user specifies four bedrooms with two bedrooms on the ground floor and two bedrooms on the first floor, the system should preserve this distribution while generating the house model.

After successful validation, the structured house requirements are passed through the backend system to the **Blender-based procedural 3D generation module**. Blender is used to

automatically construct the conceptual house model using Python-based procedural generation techniques. Based on the structured requirements, the system can generate components such as floors, rooms, walls, doors, windows, stairs, balconies, and other supported architectural elements.

The system uses a **React-based frontend** to provide an interactive interface for entering house requirements and viewing the processing status and generated results. The backend manages communication between the user interface, requirement processing module, validation system, and Blender generation module. This creates a continuous workflow from natural-language input to a conceptual three-dimensional house model.

The proposed system aims to reduce the complexity and manual effort involved in initial 3D house modelling. It demonstrates how artificial intelligence, natural-language processing, web technologies, and procedural 3D modelling can be combined to provide an accessible approach to conceptual house visualization. The generated model is intended for **conceptual design and visualization purposes** and should not be considered a professional architectural, structural, or construction-certified design.

Overall, the AI-Based Text-to-3D House Generation System provides an intelligent workflow that bridges the gap between simple textual house requirements and three-dimensional visualization. It enables users to express their design requirements naturally while the system handles requirement extraction, validation, layout processing, and procedural 3D model generation.

1.2 LITERATURE SURVEY

The development of AI-based 3D house generation systems is supported by research and technological advancements in several areas, including Artificial Intelligence, Natural Language Processing, text-to-3D generation, procedural modelling, and computer-aided architectural design. The following literature survey discusses the major concepts related to the proposed AI-Based Text-to-3D House Generation System.

1.2.1 Artificial Intelligence in Design Automation

Artificial Intelligence has increasingly been used to automate tasks that traditionally require

significant human effort and domain knowledge. In design-related applications, AI can analyse user requirements, identify patterns, and generate structured outputs based on natural-language descriptions. AI-assisted design systems can therefore reduce repetitive manual work and provide faster initial design solutions.

In the context of house design, AI can be used to understand requirements such as the number of bedrooms, number of floors, room types, and other user preferences. Instead of requiring users to manually define every design element, an intelligent system can interpret the requirements and convert them into a structured representation. This concept forms the foundation of the proposed system, where AI-assisted processing is used to understand natural-language house descriptions before the 3D generation stage.

1.2.2 Natural Language Processing for Requirement Understanding

Natural Language Processing (NLP) enables computer systems to process and understand information expressed through human language. NLP techniques are widely used for extracting meaningful information from textual input and converting unstructured language into structured data.

For a house-generation application, users may describe their requirements using different sentence structures. For example, a user may specify *"I need a two-floor 4BHK house with two bedrooms downstairs and two bedrooms upstairs."* The system needs to identify the BHK value, number of floors, bedroom count, and floor-wise distribution from this description.

The proposed system applies AI-assisted requirement parsing to identify relevant house parameters and represent them in a structured format. This structured representation makes the requirements easier to validate and subsequently use for procedural 3D generation.

.

1.2.3 Text-to-3D Generation

Text-to-3D generation is an emerging area that focuses on producing three-dimensional representations from textual descriptions. Advances in AI have made it possible to transform descriptive information into visual or geometric representations for different applications.

Traditional 3D modelling requires users to manually create and arrange objects. Text-based generation provides an alternative approach by allowing users to describe the required object or environment using natural language. For architectural applications, this concept can be used

to convert descriptions of houses and rooms into corresponding three-dimensional structures.
The proposed system applies the text-to-3D concept specifically to conceptual house generation. Instead of directly generating an unrestricted 3D object from text, the system first converts the textual requirements into structured house information. This intermediate representation helps maintain important constraints such as BHK count, floor count, and floor-wise room distribution. Artificial Intelligence has increasingly been used to automate tasks that traditionally require significant human effort and domain knowledge. In design-related applications, AI can analyse user requirements, identify patterns, and generate structured outputs based on natural-language descriptions. AI-assisted design systems can therefore reduce repetitive manual work and provide faster initial design solutions.
In the context of house design, AI can be used to understand requirements such as the number of bedrooms, number of floors, room types, and other user preferences. Instead of requiring users to manually define every design element, an intelligent system can interpret the requirements and convert them into a structured representation. This concept forms the foundation of the proposed system, where AI-assisted processing is used to understand natural-language house descriptions before the 3D generation stage.

1.2.4 Natural Language Processing for Requirement Understanding

Natural Language Processing (NLP) enables computer systems to process and understand information expressed through human language. NLP techniques are widely used for extracting meaningful information from textual input and converting unstructured language into structured data.

For a house-generation application, users may describe their requirements using different sentence structures. For example, a user may specify *"I need a two-floor 4BHK house with two bedrooms downstairs and two bedrooms upstairs."* The system needs to identify the BHK value, number of floors, bedroom count, and floor-wise distribution from this description. The proposed system applies AI-assisted requirement parsing to identify relevant house parameters and represent them in a structured format. This structured representation makes the requirements easier to validate and subsequently

1.2.5 Procedural 3D Modelling

Procedural modelling is a technique in which three-dimensional objects are generated

automatically using predefined rules, parameters, and algorithms. It reduces the need to manually model every individual component and is particularly useful when similar structures need to be generated with different configurations.

In architectural modelling, procedural techniques can be used to generate walls, rooms, floors, doors, windows, stairs, and other components based on input parameters. Changing the input values can result in a different model without requiring the entire modelling process to be repeated manually.

The proposed system uses Blender and Python-based procedural generation to construct the conceptual house. The structured requirements received from the processing system act as parameters for the generation process. Based on these parameters, the system can determine the required number of floors, rooms, and other supported components and construct the corresponding 3D scene.

1.2.6 AI-Assisted Architectural Visualization

Computer-aided architectural design and 3D visualization have significantly improved the way building concepts are planned and presented. Three-dimensional visualization allows users to understand spatial arrangements and obtain a clearer representation of a proposed design than conventional textual or two-dimensional descriptions alone.

AI-assisted architectural visualization can further simplify the initial design process by allowing users to express requirements in natural language. Combining AI-based requirement understanding with procedural 3D modelling can provide a workflow in which textual requirements are transformed into an interactive conceptual representation.

The proposed **AI-Based Text-to-3D House Generation System** combines these concepts by connecting natural-language requirement processing with procedural Blender generation. The system is designed to provide a faster and more accessible method for producing conceptual house models while reducing the amount of manual 3D modelling required from the user.

1.3 PROPOSED SYSTEM OBJECTIVES AND SCOPES

1.3.1 Objectives of the Proposed System

The primary objective of the **AI-Based Text-to-3D House Generation System** is to develop

an intelligent and user-friendly platform that converts natural-language house requirements into a structured representation and subsequently generates a conceptual three-dimensional house model. The system aims to simplify the initial house design and visualization process by reducing the need for manual 3D modelling and specialized technical knowledge.

The major objectives of the proposed system are:

1. **To accept natural-language house requirements** from users in a simple and understandable format.
2. **To analyse and interpret user requirements** using an AI-assisted requirement parsing mechanism.
3. **To identify important house parameters** such as BHK type, number of bedrooms, number of floors, room types, and other supported requirements.
4. **To convert unstructured textual requirements into structured house data** that can be processed by the system.
5. **To correctly handle BHK requirements**, where values such as 2BHK, 3BHK, and 4BHK represent the required number of bedrooms.
6. **To preserve floor-wise requirements and constraints** specified by the user. For example, if a user specifies two bedrooms on the ground floor and two bedrooms on the first floor for a 4BHK house, the system should maintain this distribution during generation.
7. **To validate the extracted requirements** before starting the 3D generation process and identify missing, invalid, or contradictory requirements.
8. **To generate a conceptual house layout** based on the validated structured requirements.
9. **To use Blender and procedural 3D modelling techniques** to automatically generate house components such as floors, rooms, walls, doors, windows, stairs, balconies, and other supported elements.
10. **To integrate the frontend, backend, AI processing, validation, and Blender generation modules** into a unified workflow.
11. **To reduce the time and complexity of manual house modelling** during the initial conceptual design stage.
12. **To provide a visual three-dimensional representation** that helps users understand the proposed house arrangement more easily.

Overall, the system aims to demonstrate how artificial intelligence, natural-language processing, web technologies, and procedural 3D modelling can be combined to create an

accessible conceptual house-generation workflow.

1.3.2 Scope of the Proposed System

The scope of the **AI-Based Text-to-3D House Generation System** covers the development of a web-based platform that accepts natural-language house requirements and transforms them into a conceptual three-dimensional house model. The system focuses on simplifying the initial design and visualization process for users who may not have advanced architectural or 3D modelling skills.

The system allows users to provide requirements such as the number of bedrooms, BHK type, number of floors, room requirements, and supported floor-wise constraints through a natural-language interface. The AI-assisted requirement parser processes the input and extracts the relevant information into a structured representation.

The scope also includes requirement validation to ensure that the extracted information is logically consistent before model generation. The validation process is particularly useful for checking requirements involving the number of bedrooms, number of floors, and floor-wise room distribution.

After validation, the structured requirements are transferred to the backend and used by the Blender generation module. Blender uses procedural modelling techniques to construct the conceptual house based on the provided parameters. The generated model may contain architectural elements such as rooms, walls, floors, doors, windows, stairs, and balconies depending on the supported requirements.

The project also covers the integration of a **React-based frontend** with the Python-based backend and Blender generation workflow. The frontend provides the user interface for entering requirements and viewing the generated results, while the backend manages processing and communication between the different system components.

The proposed system is mainly intended for **conceptual house planning, visualization, and design assistance**. It can help users quickly visualize an initial house concept without manually modelling every component from the beginning.

However, the scope of the system does not include professional architectural certification, Artificial Intelligence has increasingly been used to automate tasks that traditionally require significant human effort and domain knowledge. In design-related applications, AI can analyse user requirements, identify patterns, and generate structured outputs based on natural-language descriptions. AI-assisted design systems can therefore reduce repetitive manual work and

provide faster initial design solutions.

In the context of house design, AI can be used to understand requirements such as the number of bedrooms, number of floors, room types, and other user preferences. Instead of requiring users to manually define every design element, an intelligent system can interpret the requirements and convert them into a structured representation. This concept forms the foundation of the proposed system, where AI-assisted processing is used to understand natural-language house descriptions before the 3D generation stage.

1.2.2 Natural Language Processing for Requirement Understanding

Natural Language Processing (NLP) enables computer systems to process and understand information expressed through human language. NLP techniques are widely used for extracting meaningful information from textual input and converting unstructured language into structured data.

For a house-generation application, users may describe their requirements using different sentence structures. For example, a user may specify *"I need a two-floor 4BHK house with two bedrooms downstairs and two bedrooms upstairs."* The system needs to identify the BHK value, number of floors, bedroom count, and floor-wise distribution from this description.

The proposed system applies AI-assisted requirement parsing to identify relevant house parameters and represent them in a structured format. This structured representation makes the requirements easier to validate and subsequently structural engineering calculations, construction approval, detailed structural analysis, or legally approved building plans. The generated model should therefore be considered a conceptual 3D representation rather than a final construction-ready architectural design.

Future versions of the system can extend the current scope by supporting more advanced architectural layouts, furniture placement, realistic materials, lighting and rendering, roof variations, parking areas, terrain, interactive 3D editing, and additional export formats.

CHAPTER 2

REQUIREMENTS SPECIFICATIONS

Requirements Specification is a detailed description of the requirements, functions, constraints, and expected behavior of a software system. It defines what the system should do, how users interact with it, and the conditions that the system must satisfy.

In this project, the Requirements Specification describes the functional and non-functional requirements of the AI-Based Text-to-3D House Generation System. It specifies how the system accepts natural-language house requirements, processes them using AI, validates the requirements, generates structured house data, and produces a conceptual 3D house model using Blender.

It also covers the software and hardware requirements, user requirements, security requirements, external interfaces, system features, and operating constraints necessary for developing and operating the proposed system**.**

2.1 OVERALL DESCRIPTION

The **AI-Based Text-to-3D House Generation System** is designed as an intelligent software system that converts natural-language house requirements into a structured house specification and generates a conceptual three-dimensional house model using Blender. The system combines a React-based user interface, a Python-based backend, AI-assisted requirement processing, validation mechanisms, and procedural 3D generation.

The user interacts with the system through a web interface and provides a description of the desired house. The input may contain information such as BHK type, number of floors, number of bedrooms, room requirements, and floor-wise room distribution. The system analyses this input and extracts the relevant requirements.

The extracted information is converted into a structured representation that can be understood by the backend and Blender generation module. Before generating the model, the system validates the requirements to identify inconsistencies or invalid information. This helps ensure that important requirements, such as the total number of bedrooms and their distribution across floors, are maintained during 3D generation.

After successful validation, the structured house specification is processed by the procedural generation module. Blender uses Python-based scripts to create the required architectural components and arrange them according to the provided requirements. The resulting model

represents a conceptual visualization of the requested house.

The overall system is divided into several interconnected components:

- **User Interface:** Accepts natural-language house requirements.
- **AI Requirement Parser:** Extracts meaningful house information from the user's description.
- **Structured Requirement Module:** Represents the extracted information in a machine-readable format.
- **Validation Module:** Checks the consistency of the requirements.
- **Backend API:** Manages communication between the frontend, processing modules, and Blender.
- **Blender Generation Module:** Creates the conceptual 3D house procedurally.
- **Output Module:** Provides the generated result to the user.

The system is intended to simplify the initial house visualization process and reduce the amount of manual effort required for basic 3D modelling.

2.1.1 Functional Requirements:

The functional requirements define the major operations that the proposed system must perform.

1. User Input

The system shall allow the user to enter house requirements using natural language through a text input interface.

2. Requirement Processing

The system shall process the user's textual description and identify relevant house requirements.

3. BHK Detection

The system shall identify BHK information from the user input.

For example:

4BHK → 4 bedrooms

The system should support common variations such as:

- 2BHK
- 3BHK
- 4BHK
- 5BHK

- two-bedroom house
- four-bedroom house

4. Floor Detection

The system shall identify the required number of floors from the user's description.

For example:

"Create a house with 2 floors."

The system should identify the requirement as:

Total Floors = 2

5. Room Requirement Detection

The system shall identify supported room requirements such as:

- Bedrooms
- Living room
- Kitchen
- Dining room
- Bathrooms
- Family living area
- Balcony
- Stairs
- Other supported features

6. Floor-Wise Requirement Processing

The system shall identify and preserve room requirements associated with particular floors.

For example:

"2 bedrooms on the ground floor and 2 bedrooms on the first floor."

The system should maintain:

- Ground Floor → 2 Bedrooms
- First Floor → 2 Bedrooms
- Total → 4 Bedrooms

7. Structured Data Generation

The system shall convert the extracted requirements into a structured data format such as JSON for further processing.

8. Requirement Validation

The system shall validate the structured requirements before starting 3D generation.

The validation process should identify:

- Missing information
- Invalid values
- Contradictory requirements
- Incorrect room counts
- Inconsistent floor distribution

9. House Layout Generation

The system shall generate a basic conceptual arrangement of rooms based on the validated requirements.

10. 3D House Generation

The system shall generate a conceptual 3D house model using Blender and procedural modelling techniques.

11. Architectural Component Generation

The Blender module shall generate supported components such as:

- Floors
- Rooms
- Walls
- Doors
- Windows
- Stairs
- Balconies

12. Multi-Floor Generation

The system shall generate multiple floors according to the specified floor count and position them correctly in the 3D scene.

13. Result Generation

The system shall provide the generated 3D house output after successful processing.

14. Error Handling

The system shall display meaningful error messages when the provided requirements are invalid, incomplete, contradictory, or when generation fails.

15. Status Feedback

The system shall provide appropriate processing feedback during different stages, such

- Requirement processing
- Validation
- 3D generation
- Completion

.

2.2 PRODUCT PERSPECTIVE

The **AI-Based Text-to-3D House Generation System** is designed as a web-based application that integrates Artificial Intelligence, a backend processing system, and Blender-based procedural 3D modelling. The system provides an interface through which users can enter their house requirements in natural language.

The system acts as an integrated solution where the user's requirements are processed and converted into structured information before generating the 3D house model. The major components of the system include the **React-based frontend, AI requirement processing module, validation module, Python backend, and Blender 3D generation module**.

The frontend is responsible for collecting user requirements and displaying the processing status and generated results. The AI processing module identifies important requirements such as **BHK type, number of floors, rooms, and floor-wise constraints** from the user's text. The validation module checks whether the extracted requirements are consistent and suitable for generation.

The backend manages communication between the frontend, requirement-processing modules, and Blender. Blender is used to procedurally generate the conceptual 3D house based on the validated structured requirements. The generated model is then provided as the final visual output to the user.

The overall perspective of the proposed system can be represented as:

User → React Frontend → AI Requirement Parser → Validation → Python Backend → Blender → 3D House Output

The system is intended to reduce manual modelling effort and provide a faster way to visualize house requirements. It is primarily designed as a **conceptual design and visualization system**, rather than a replacement for professional architectural or structural design software.

-

2.3 PRODUCT FUNCTIONS

The AI-Based Text-to-3D House Generation System performs a series of functions to convert user-provided house requirements into a conceptual 3D house model. The major functions of

the system are described below**.**

2.3.1 User Requirement Input

The system allows users to enter house requirements using natural language. The input may include details such as the number of bedrooms, BHK type, number of floors, rooms, and specific floor-wise requirements.

2.3.2 AI-Based Requirement Processing

The system processes the user's natural-language input using an AI-assisted requirement parser. It identifies and extracts important information required for house generation.

2.3.3 House Requirement Extraction

The system extracts parameters such as:

- BHK type
- Number of bedrooms
- Number of floors
- Required rooms
- Floor-wise room distribution
- Other specified house requirements

.

2.3.4 Requirement Structuring

The extracted information is converted into a structured format that can be understood by the backend and Blender generation module.

2.3.5 Requirement Validation

The system validates the extracted requirements before starting 3D generation. It checks important constraints such as the total number of bedrooms and the specified distribution of rooms across floors..

2.3.6 3D House Generation

After successful validation, the structured requirements are passed to the 3D generation process. Blender uses the provided parameters to procedurally create the conceptual house model.

2.3.7 Architectural Component Generation

The system can generate basic architectural elements required for the house model, such as **floors, rooms, walls, doors, windows, stairs, and balconies**, based on the supported requirements.

2.3.8 Multi-Floor House Generation

The system supports the generation of houses with multiple floors. Floor-wise requirements are considered during the generation process so that specified rooms can be placed on their corresponding floors.

2.3.9 Result Visualization

The generated 3D house model is provided as the final visual output, allowing the user to understand and visualize the proposed house structure.

2.3.10 Error Handling and Feedback

If the entered requirements are incomplete, inconsistent, or cannot be processed, the system provides appropriate feedback instead of proceeding with an invalid generation request.

.

2.4 USER CHARACTERISTICS

The AI-Based Text-to-3D House Generation System is designed to be simple and user-friendly so that users can provide house requirements without requiring advanced knowledge of 3D modelling or Blender.

2.4.1 General Users

The system can be used by individuals who want to generate a conceptual 3D representation of a house based on their requirements. Users should be able to describe their requirements using simple natural language.

2.4.2 Required Knowledge

Users do not need professional architectural or 3D modelling knowledge. Basic knowledge of house requirements, such as **BHK type, number of bedrooms, number of floors, and room requirements**, is sufficient to interact with the system.

2.4.3 User Interaction

The user enters the required house specifications through the frontend interface. The system processes the input, validates the requirements, and generates the corresponding conceptual 3D house model..

2.4.4 Technical Knowledge

No advanced programming knowledge is required for normal users. The technical processing, AI-based requirement extraction, validation, backend communication, and Blender-based 3D generation are handled by the system.

2.4.5 Expected User Behavior

Users are expected to provide clear and meaningful house requirements. If the input contains conflicting, incomplete, or unsupported requirements, the system may request correction or provide an appropriate validation message.

2.5 OPERATING ENVIRONMENT CONSTRAINTS

The **AI-Based Text-to-3D House Generation System** requires a suitable software and hardware environment for processing user requirements, running the backend services, and generating the 3D house model using Blender. The operating environment should provide sufficient resources for smooth communication between the frontend, backend, AI processing

module, and 3D generation module.

The major operating environment constraints are as follows:

- The system requires a modern computer capable of running the frontend, backend, and Blender.
- A suitable web browser is required to access the user interface.
- The backend environment must support the required Python-based services and dependencies.
- Blender must be installed and configured for procedural 3D house generation.
- Sufficient processing power and memory are required for 3D model generation.
- The system requires proper communication between the frontend, backend, and Blender components.
- Internet connectivity may be required for AI-assisted requirement processing if the implementation uses an external AI service.
- The generated 3D model quality and processing time may depend on the available hardware resources.

2.5.1 Software Specifications

The software requirements for the proposed system include:

SL. NO.	SOFTWARE	PURPOSE
1	React.js	Frontend development
2	HTML	Structure of the web interface
3	CSS	Styling and layout
4	JavaScript	Frontend functionality
5	Python	Backend and processing
6	Blender	Procedural 3D house generation
7	Web Browser	Accessing the application
8	Required Python Libraries	Backend and AI processing

2.5.2 Hardware Specifications

The recommended hardware requirements for the system include:

SL. NO.	HARDWARE	RECOMMENDED REQUIREMENT
1	Processor	Intel Core i5 / equivalent or above
2	RAM	8 GB or above
3	Storage	10 GB or more free space
4	Graphics	Dedicated GPU recommended for better 3D performance
5	Display	Standard HD display or above
6	Internet	Required when external AI services are used

2.6 SPECIFIC REQUIREMENTS

The specific requirements define the detailed functional, non-functional, and security requirements of the **AI-Based Text-to-3D House Generation System**. These requirements describe the services expected from the system and the conditions under which the system should operate.

2.6.1 User Requirements:

The system shall provide the following requirements for users:

1. The user shall be able to enter house requirements in natural language.
2. The user shall be able to specify the **BHK type and number of bedrooms**.
3. The user shall be able to specify the required number of floors.
4. The user shall be able to provide room requirements.
5. The user shall be able to specify floor-wise room requirements when required.
6. The system shall process the entered requirements using the AI-assisted requirement parser.
7. The system shall display validation feedback for invalid or conflicting requirements.
8. The system shall generate a conceptual 3D house based on the validated requirements.
9. The user shall be able to view the generated 3D house output through the system.

2.6.2 Admin Requirements:

The proposed system primarily focuses on user-driven house generation. If administrative

functionality is included in the implementation, an administrator may be responsible for:

1. Managing system configuration and supported requirements.
2. Maintaining the backend and generation services.
3. Monitoring system errors and processing failures.
4. Managing required software components and dependencies.
5. Ensuring that the AI processing and Blender generation modules operate correctly.

2.6.3 Functional Requirements:

he major functional requirements of the system are:

- **FR1:** The system shall accept natural-language house requirements.
- **FR2:** The system shall identify relevant house parameters from the input.
- **FR3:** The system shall identify BHK and bedroom requirements.
- **FR4:** The system shall identify the required number of floors.
- **FR5:** The system shall identify specified rooms and floor-wise requirements.
- **FR6:** The system shall convert extracted requirements into structured house data.
- **FR7:** The system shall validate the structured requirements.
- **FR8:** The system shall preserve explicitly specified floor-wise constraints.
- **FR9:** The system shall send validated requirements to the 3D generation process.
- **FR10:** Blender shall generate the supported architectural components procedurally.
- **FR11:** The system shall provide the generated 3D house as output.
- **FR12:** The system shall provide suitable error messages when requirements cannot be processed.

2.6.4 Non- Functional Requirements:

The system should satisfy the following non-functional requirement

Performance

The system should process user requirements and generate the 3D model within a reasonable time depending on the complexity of the input and available hardware.

Usability

The user interface should be simple and understandable so that users can provide requirements without advanced technical knowledge.

Reliability

The system should validate requirements before starting the generation process to reduce incorrect or inconsistent outputs.

Maintainability

The frontend, backend, AI processing, validation, and Blender generation components should be organized so that individual modules can be maintained and improved.

Scalability

The system should allow future extensions such as improved room layouts, furniture generation, materials, rendering, and additional customization features.

Compatibility

The system should operate with supported modern web browsers and the required software environment.

.

2.6.5 Security Requirements:

The system should follow basic security practices to protect the application and its processing environment.

1. User inputs should be validated before processing.
2. Invalid or malicious input should not be directly executed as system commands.
3. Backend endpoints should validate incoming requests.
4. Sensitive configuration information should not be exposed through the frontend.
5. Communication between system components should be appropriately controlled.
6. Generated files and temporary data should be handled securely.
7. External AI services, if used, should be accessed using secure configuration and authentication mechanisms.

2.7 EXTERNAL INTERFACE REQUIREMENTS

2.7 EXTERNAL INTERFACE REQUIREMENTS

The **AI-Based Text-to-3D House Generation System** requires communication between the user interface, backend services, AI-based requirement processing module, and Blender generation module. The external interface requirements define how these components interact

with the system and how information is exchanged.

2.7.1 User Interface

The system shall provide a web-based user interface through which users can:

- Enter house requirements in natural language.
- Submit the requirements for processing.
- View the extracted or structured house requirements.
- Receive validation messages and processing status.
- View the generated conceptual 3D house model.

The interface should be simple, responsive, and easy to understand.

2.7.2 Frontend–Backend Interface

The React frontend communicates with the Python backend through appropriate API requests. The frontend sends the user's house requirements to the backend and receives the processed requirements, validation status, errors, and generation results.

2.7.3 AI Processing Interface

The backend communicates with the AI-assisted requirement processing module to extract relevant information from the user's natural-language input. The processed information is converted into a structured representation suitable for validation and 3D generation.

2.7.4 Backend–Blender Interface

The validated structured house requirements are passed from the backend to the Blender generation module. Blender uses these parameters to create the required conceptual 3D house components.

2.7.5 Output Interface

The system provides the generated 3D house model as the final output. The output should represent the validated requirements provided by the user, including supported details such as the number of bedrooms, floors, rooms, and specified floor-wise distribution.

2.7.6 Communication Interface

The system uses web-based communication between the frontend and backend. API-based communication is used for transferring user requirements and generation-related data between the different system components.

2.8 SYSTEM FEATURES

2.8 SYSTEM FEATURES

The **AI-Based Text-to-3D House Generation System** provides several features that

enable users to convert natural-language house requirements into a conceptual 3D house model.

2.8.1 Natural Language House Requirement Input

The system allows users to describe their house requirements using natural language. Users can specify requirements such as BHK type, number of bedrooms, number of floors, rooms, and other supported constraints.

2.8.2 AI-Based Requirement Understanding

The AI-assisted processing module analyzes the user's input and identifies important house-related information. This reduces the need for users to enter every parameter manually in a structured form.

2.8.3 Structured House Specification

The extracted requirements are converted into a structured format containing the parameters required for validation and 3D model generation.

2.8.4 Requirement Validation

Before generating the model, the system checks the extracted requirements for consistency. It verifies important constraints such as bedroom count, floor count, and explicitly specified floor-wise room distribution.

2.8.5 Procedural 3D House Generation

The validated house specification is passed to Blender, where the house is generated procedurally. Basic architectural elements such as floors, rooms, walls, doors, windows, stairs, and balconies can be generated according to the supported requirements.

2.8.6 Multi-Floor Support

The system supports multi-floor house generation. Requirements specified for individual floors are considered during the generation process.

2.8.7 Frontend and Backend Integration

The React frontend provides the user interface, while the Python backend manages requirement processing, validation, and communication with the 3D generation module.

2.8.8 3D Visualization

The final output is a conceptual 3D representation of the requested house, allowing users to visually understand the proposed structure without manually creating the model in Blender.

2.8.9 Error and Status Feedback

The system provides appropriate feedback when requirements are invalid, incomplete, conflicting, or when an error occurs during processing or generation.

.

CHAPTER 3
SYSTEM DESIGN AND TEST PLAN

This chapter describes the design of the **AI-Based Text-to-3D House Generation System** and the testing methods used to verify its functionality. The system design explains the flow of information between the user interface, AI requirement processing module, validation module, backend, and Blender-based 3D generation module.

The chapter includes the **Data Flow Diagrams (DFD), Use Case Diagram, data structure design, performance requirements, software quality attributes, detailed system design, and test cases**. The test plan is prepared to verify that the system correctly processes natural-language house requirements, maintains specified constraints, and generates the expected conceptual 3D house output.

3.1 DATA FLOW DIAGRAM (DFD)

The **Data Flow Diagram (DFD)** represents the flow of information between the user and the different modules of the **AI-Based Text-to-3D House Generation System**. It shows how the user's natural-language house requirements are received, processed, validated, and converted into a conceptual 3D house model.

The main data flow of the proposed system is:

User → House Requirement Input → AI Requirement Parser → Structured House Data → Validation Module → Backend → Blender 3D Generation → 3D House Output

The DFD helps to understand the movement and processing of data within the system. It also shows the interaction between the frontend, requirement processing, validation, backend, and Blender modules.

.

3.1.1 Level 0 DFD

The **Level 0 DFD**, also known as the context-level diagram, represents the complete system as a single process. The user provides house requirements as input to the system, and the system processes the requirements and generates a conceptual 3D house as output.

Input: Natural-language house requirements from the user.

Process: AI-Based Text-to-3D House Generation System.

Output: Validated house information and generated conceptual 3D house model.

Figure 3.1: DFD Level 0

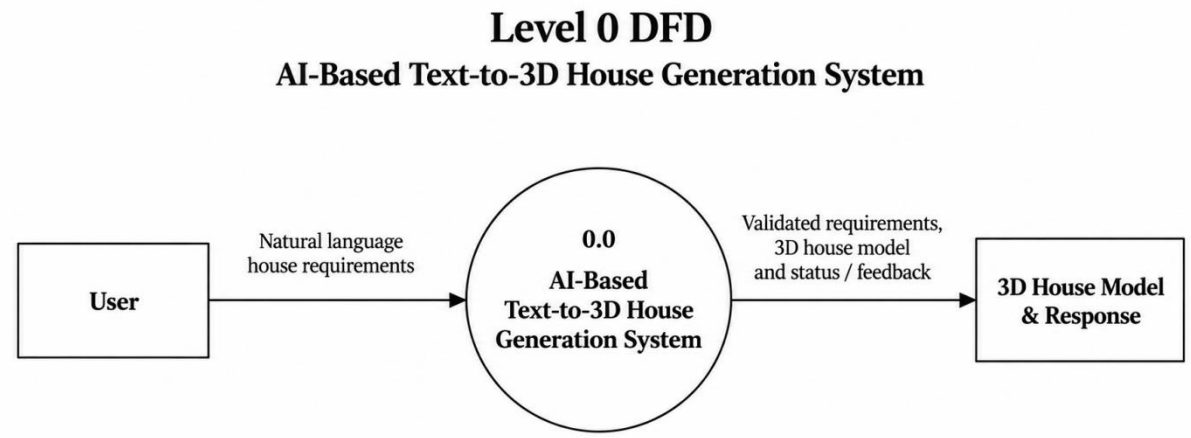


Figure 3.1: DFD Level 0

3.1.2 Level 1 DFD

The **Level 1 DFD** provides a detailed representation of the internal processes of the proposed system. The user's natural-language input is first received by the frontend and then passed to the AI requirement processing module.

The AI module extracts relevant information such as **BHK, number of bedrooms, number of floors, rooms, and floor-wise requirements**. The extracted information is converted into structured house data and passed to the validation module.

After successful validation, the structured requirements are processed by the backend and passed to the Blender generation module. Blender then generates the conceptual 3D house based on the validated requirements. The generated output is returned to the user through the application interface.

.

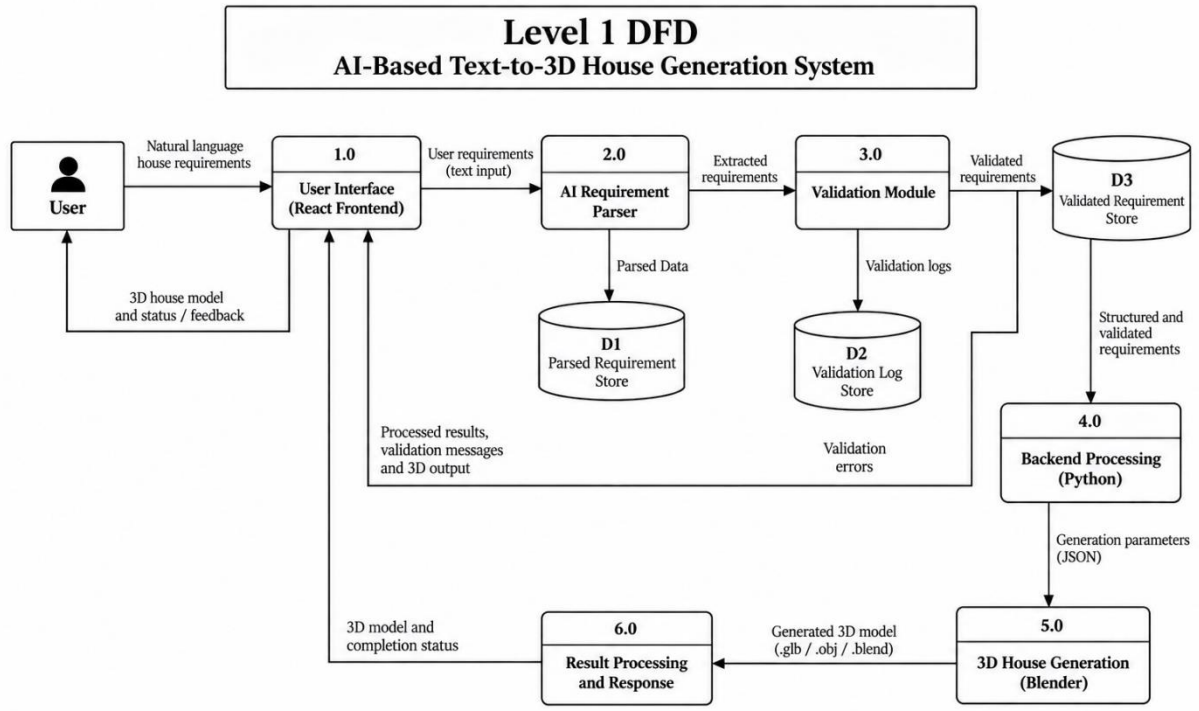


Figure 3.2: DFD Level 1

3.2 USE CASE DIAGRAM

The **Use Case Diagram** represents the interaction between the user and the **AI-Based Text-to-3D House Generation System**. It identifies the major activities performed by the user and the services provided by the system.

The proposed system primarily has one main actor, **User**. The user interacts with the system by entering natural-language house requirements, submitting them for processing, viewing

the extracted requirements, checking validation results, and viewing the generated 3D house model.

The major use cases of the system are:

1. **Enter House Requirements** – The user provides house requirements in natural language.
2. **Submit Requirements** – The user submits the entered requirements for processing.
3. **Process Requirements** – The system analyzes the input and extracts relevant house parameters.
4. **View Structured Requirements** – The user can view the extracted house information.
5. **Validate Requirements** – The system checks the requirements for consistency and constraint violations.
6. **Generate 3D House** – The validated requirements are used to generate the conceptual 3D house using Blender.
7. **View 3D House Output** – The user views the generated 3D house model.
8. **View Status and Feedback** – The user receives processing status, validation messages, and error feedback when required.

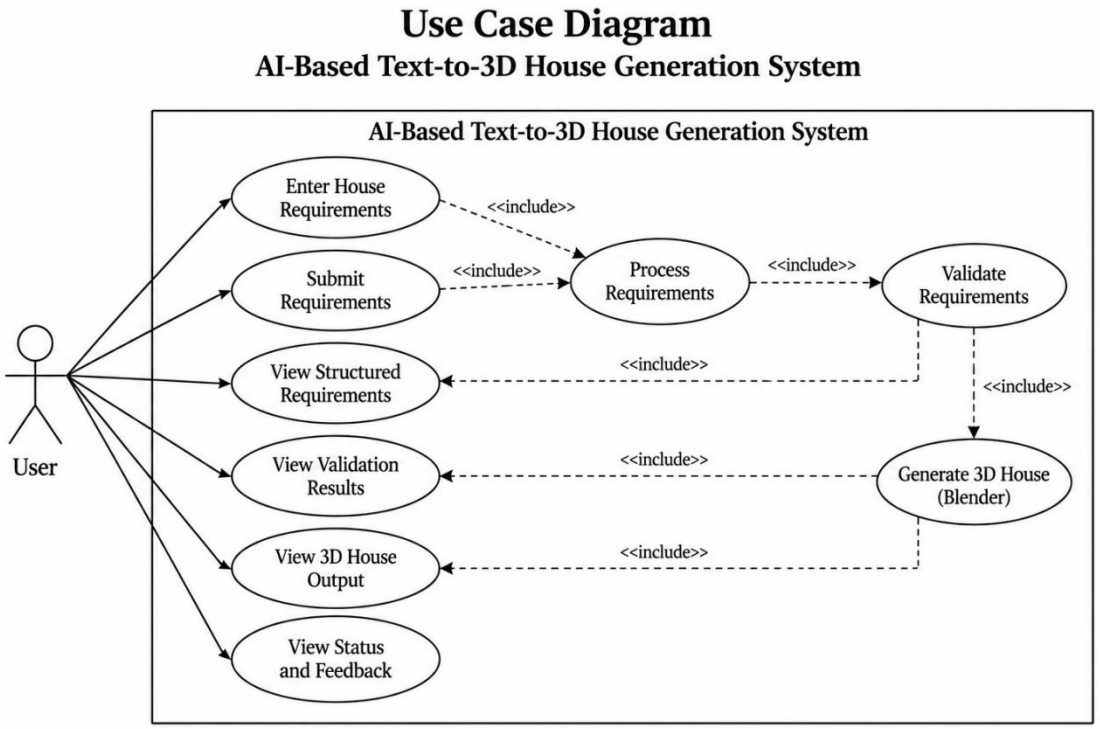


Figure 3.3: User Use Case Diagram

3.3 TABLE DESIGNS

The Table Design section describes the structured data used by the AI-Based Text-to-3D House Generation System. The system converts natural-language requirements into structured house information before validation and 3D generation.

Since the proposed system primarily processes house requirements rather than relying on a conventional relational database, the table design represents the major fields used to organize the house-generation data**.**

3.3.1 HOUSE REQUIREMENT DATA STRUCTURE

- Stores core user identity, authentication credentials, and role-related information for patients, doctors, hospitals, and administrators within the healthcare platform.

Table 3.1: auth_user table

FIELD NAME	DATA TYPE	DESCRIPTION
House_Type	String	Specifies the type of house such as BHK-based house
BHK	String	Specifies the BHK requirement
Bedrooms	Integer	Total number of bedrooms required
Floors	Integer	Total number of floors
Rooms	Array/List	Contains the required rooms
Floor_Wise_Rooms	Object/List	Specifies room distribution across individual floors

Bathrooms	Integer	Number of bathrooms required, when specified
Kitchen	Integer	Number of kitchens required, when specified
Living_Room	Integer	Number of living rooms required, when specified
Additional_Requirements	String/List	Contains other supported requirements
Validation_Status	String	Indicates whether the requirements are valid
Generation_Status	String	Indicates the status of 3D model generation

3.3.2 DATA PROCESSING

The structured house data is generated after processing the user's natural-language input. The **AI Requirement Parser** extracts the relevant parameters and converts them into the above structured representation. The **Validation Module** then checks the information before it is passed to the backend and Blender.

For example, an input such as **"Generate a 4BHK house with 2 floors, with 2 bedrooms on the ground floor and 2 bedrooms on the first floor"** should be represented with **4 total bedrooms**, **2 floors**, and the explicitly specified **2+2 floor-wise distribution**.

This structured representation provides a consistent input format for the subsequent validation and 3D house generation processes.

3.4 PERFORMANCE REQUIREMENTS

The **AI-Based Text-to-3D House Generation System** should provide efficient processing of user requirements and generate the conceptual 3D house within a reasonable amount of time. The performance of the system depends on the complexity of the house requirements, number of floors, number of rooms, AI processing time, Blender generation time, and available hardware resources.

The major performance requirements are as follows:

3.4.1 Requirement Processing

The system should process the user's natural-language house requirements efficiently

and extract the required parameters such as BHK, bedrooms, floors, rooms, and floor-wise constraints.

3.4.2 Validation Performance

The validation module should verify the extracted requirements before 3D generation. Validation should be completed without unnecessary processing delays.

3.4.3 3D Generation Performance

The Blender generation module should generate the conceptual 3D house based on the validated structured requirements. Generation time may vary depending on the complexity of the requested house and the available system resources.

3.4.4 Response Time

The frontend should provide appropriate processing status or feedback while the requirements are being processed and the 3D model is being generated.

3.4.5 Resource Utilization

The system should use CPU, RAM, storage, and GPU resources efficiently. Since 3D generation can require significant computational resources, sufficient hardware should be available for smooth operation.

3.4.6 Scalability

The system should be designed so that additional features such as more complex room layouts, furniture, materials, rendering, and customization can be added without requiring a complete redesign of the system.

3.4.7 Reliability During Processing

The system should handle invalid requirements and generation errors appropriately. It should provide meaningful feedback instead of producing an incorrect or incomplete result whenever possible.

.

3.5 SOFTWARE QUALITY ATTRIBUTES

The **AI-Based Text-to-3D House Generation System** should satisfy a set of software quality attributes to ensure that the system is reliable, usable, maintainable, and efficient. These attributes define the overall quality expected from the proposed system.

3.5.1 Usability

The system should provide a simple and user-friendly interface. Users should be able to enter

house requirements in natural language without requiring advanced knowledge of programming, Blender, or 3D modelling.

3.5.2 Reliability

The system should process user requirements consistently and validate the extracted information before generating the 3D house. Invalid or conflicting requirements should be handled appropriately.

3.5.3 Performance

The system should process requirements and generate the conceptual 3D model efficiently. The processing time may vary depending on the complexity of the house and the available hardware resources.

3.5.4 Maintainability

The system should be organized into separate modules such as frontend, AI requirement processing, validation, backend, and Blender generation. This modular structure makes it easier to identify errors and implement future changes.

3.5.5 Scalability

The system should support future improvements without major changes to the existing architecture. Additional features such as furniture generation, improved layouts, materials, rendering, and customization can be added in future versions.

3.5.6 Flexibility

The system should be capable of handling different natural-language descriptions of house requirements. It should support variations in requirements such as different BHK types, floor counts, rooms, and supported floor-wise constraints.

3.5.7 Security

User inputs and system resources should be handled securely. Backend interfaces should validate incoming data, and sensitive configuration information should not be exposed to users.

3.5.8 Compatibility

The system should work with supported modern web browsers and the required software environment, including the Python backend and Blender 3D generation environment.

3.5.9 Accuracy of Requirement Interpretation

The system should preserve important information provided by the user during requirement processing. In particular, explicitly specified values such as **total bedrooms, number of floors, and floor-wise bedroom distribution** should be maintained correctly throughout the generation process.

3.6 DETAILED DESIGN

The **Detailed Design** describes the internal structure and working of the major modules of the **AI-Based Text-to-3D House Generation System**. The system is designed using a modular approach so that each component performs a specific function and communicates with the other components through well-defined data flows.

3.6.1 User Interface Module

The User Interface is developed using **React**. It allows the user to enter natural-language house requirements and submit them for processing.

The interface is responsible for:

- Accepting house requirements from the user.
- Sending the requirements to the backend.
- Displaying processing status.
- Displaying validation messages.
- Presenting the generated 3D house output.

3.6.2 AI Requirement Parser Module

The AI Requirement Parser processes the natural-language input provided by the user. It identifies important house-related parameters from the input and converts them into structured information.

The module can identify requirements such as:

- BHK type
- Total number of bedrooms
- Number of floors
- Required rooms
- Floor-wise room requirements
- Other supported house constraints

For example, a requirement such as **"4BHK house with 2 floors, 2 bedrooms on the ground floor and 2 bedrooms on the first floor"** is interpreted as **4 total bedrooms across 2 floors**, while preserving the specified floor-wise distribution.

3.6.3 Structured Requirement Module

The extracted information is converted into a structured representation. This provides a consistent format for the validation and generation modules.

The structured data contains fields such as:

House Type → BHK → Bedrooms → Floors → Rooms → Floor-Wise Rooms → Additional Requirements

This structured representation is passed to the validation process.

3.6.4 Validation Module

The Validation Module verifies the structured house requirements before starting 3D generation.

The module checks:

- Whether required values are available.
- Whether the BHK and bedroom information are consistent.
- Whether the number of bedrooms matches the specified requirement.
- Whether the number of floors is valid.
- Whether floor-wise room distribution is consistent.
- Whether conflicting requirements are present.

If the requirements are valid, the process continues to 3D generation. Otherwise, an appropriate validation message is returned to the user.

3.6.5 Backend Processing Module

The backend is implemented using **Python-based services**. It acts as the communication layer between the frontend, requirement-processing modules, validation system, and Blender.

The backend is responsible for:

1. Receiving user requirements.
2. Processing the request.
3. Managing structured house data.
4. Performing or coordinating validation.
5. Preparing generation parameters.
6. Communicating with the Blender generation module.
7. Returning the generation status and output to the frontend.

3.6.6 Blender 3D Generation Module

The Blender module is responsible for creating the conceptual 3D house using procedural modelling techniques.

Based on the validated house parameters, Blender can generate supported architectural elements such as:

- Floors
- Rooms
- Walls

- Doors
- Windows
- Stairs
- Balconies

The generation process uses the structured requirements as parameters instead of requiring the complete house to be manually modelled.

3.6.7 Result Processing Module

After Blender completes the generation process, the generated 3D model and generation status are handled by the result-processing component.

The module prepares the result for presentation through the frontend and provides appropriate feedback if the generation process fails or encounters an error.

3.6.8 Overall Processing Sequence

The detailed processing sequence of the system is:

Step 1: User enters natural-language house requirements.

Step 2: React frontend sends the input to the backend.

Step 3: AI Requirement Parser extracts the relevant house parameters.

Step 4: Extracted information is converted into structured house data.

Step 5: Validation Module checks the requirements and constraints.

Step 6: Validated requirements are passed to the backend generation process.

Step 7: Backend sends the required generation parameters to Blender.

Step 8: Blender procedurally generates the conceptual 3D house.

Step 9: The generated result is processed and returned to the frontend.

Step 10: The user views the generated 3D house and corresponding status or feedback.

3.7 TEST CASES

Testing is an important part of the AI-Based Text-to-3D House Generation System. It is performed to verify that the system correctly accepts user requirements, processes the natural-language input, validates the extracted information, communicates with the backend, and generates the expected conceptual 3D house.

The test cases cover the major functional modules of the system, including user input, AI requirement parsing, requirement validation, 3D house generation, frontend-backend integration, Blender generation, and system performance.

3.7.1 Test Case for Admin

Category	Text Case Description	Expected Result
User Input	Enter Valid House Requirement	House requirement is accepted successfully.
User Input	Enter Empty Requirement	Appropriate validation message is displayed..
Requirement Processing	Extract BHK Requirement.	BHK value is identified correctly.
Requirement Processing	Extract Number of Floors.	Required floor count is identified correctly.
Requirement Processing	Extract Room Requirements	Specified rooms are identified correctly.
AI Requirement Parsing	Process Natural-Language Input	Relevant house parameters are extracted correctly.
AI Requirement Parsing	Process Combined Requirements	BHK, bedrooms, floors, and rooms are extracted correctly.
Constraint Validation	Validate Bedroom Count.	Total bedroom requirement is validated correctly
Constraint Validation	Validate Floor-Wise Distribution.	Specified room distribution across floors is preserved.
Constraint Validation	Detect Conflicting Requirements	Appropriate validation message is displayed..
3D Generation	Generate Basic House.	Conceptual 3D house is generated successfully..
3D Generation	Generate Multi-Floor	Required number of floors is generated

	House.	correctly.
3D Generation	Generate Architectural Components.	Supported walls, doors, windows, and other components are generated..
Backend Integration	Send Requirements to Backend.	Requirements are received and processed successfully.
Backend Integration	Receive Generation Result	Generated result is returned to the frontend. .
Blender Generation	Generate House from Structured Data	Blender generates the house according to validated parameters.
Error Handling	Process Invalid Requirements	Appropriate error or validation message is displayed..
Performance Testing	Process House Requirements.	System processes requirements within a reasonable time..
Performance Testing	Generate 3D Model	3D model is generated within a reasonable time depending on complexity.

CHAPTER 4
IMPLEMENTATION AND RESULTS

This chapter describes the implementation of the **AI-Based Text-to-3D House Generation System** and presents the results obtained from the developed system. The implementation focuses on integrating the React-based frontend, Python-based backend, AI-assisted requirement processing, validation module, and Blender-based procedural 3D generation module.

The chapter explains how the different modules work together to convert natural-language house requirements into a structured specification and subsequently generate a conceptual 3D house model. The implementation results demonstrate the working of the major features and the output produced by the system.

4.1 IMPLEMENTATION

The proposed **AI-Based Text-to-3D House Generation System** is implemented as an integrated web-based application that combines a React frontend, a Python-based backend, AI-assisted requirement processing, validation, and Blender-based procedural 3D house generation. The implementation is divided into three major parts: **Backend Development, Frontend Development, and Integration and Testing**.

The system accepts house requirements in natural language and processes them to identify important parameters such as BHK type, number of bedrooms, number of floors, rooms, and floor-wise requirements. These requirements are converted into a structured format and validated before being passed to the 3D generation module. Blender then uses the validated information to create the conceptual 3D house model.

The implementation follows a modular approach so that each component can perform its specific function independently while communicating with the other components through defined interfaces. This approach improves maintainability, simplifies testing, and allows future extensions to the system.

4.1.1 Backend Development

The backend of the proposed system is developed using a **Python-based architecture** to handle requirement processing, validation, communication with the frontend, and interaction with the Blender-based 3D generation module.

The backend receives the house requirements submitted from the React frontend and processes the information using the AI-assisted requirement parser. Important parameters such as **BHK type, number of bedrooms, number of floors, required rooms, and floor-wise room distribution** are extracted and converted into a structured house specification.

Before starting the 3D generation process, the backend performs requirement validation. This ensures that the extracted requirements are logically consistent and that explicitly specified constraints, such as the number of bedrooms on each floor, are preserved.

After successful validation, the structured house data is passed to the Blender generation module. Blender uses this information to procedurally generate the required house structure, including supported elements such as **floors, rooms, walls, doors, windows, stairs, and balconies**.

The backend also manages the communication between the frontend and the generation module. It returns the processing status, validation results, errors when applicable, and the generated output to the frontend for display.

Main backend responsibilities are:

1. Receive house requirements from the frontend.
2. Process natural-language house requirements.
3. Extract important house parameters.
4. Convert requirements into structured data.
5. Validate house constraints and floor-wise requirements.
6. Send validated data to the Blender generation module.
7. Handle generation status and errors.
8. Return the generated result to the frontend.

4.1.2 Frontend Development

The frontend of the proposed system is developed using **React** to provide an interactive and user-friendly interface for entering house requirements and viewing the generated results. The frontend acts as the primary interaction layer between the user and the backend system.

The user can enter house requirements using natural language. The entered requirements are submitted to the backend for AI-assisted processing and validation. The frontend then displays

the processed information and provides appropriate status or validation feedback to the user. After successful processing and validation, the frontend receives the generated result from the backend and presents the **conceptual 3D house output** to the user.

Main frontend functions are:

1. Provide an interface for entering house requirements.
2. Accept natural-language descriptions of house requirements.
3. Submit user requirements to the backend.
4. Display processed and structured requirements.
5. Display validation messages and system status.
6. Receive the generated 3D house result.
7. Present the final 3D output to the user.
8. Provide a simple and user-friendly interaction experience.

4.1.3 Integration and Testing

The frontend, backend, AI-assisted requirement processing, validation module, and Blender generation module are integrated to form a complete working system. The integration ensures that information flows correctly from the user's natural-language input to the final conceptual 3D house output.

The user first enters the house requirements through the React frontend. These requirements are sent to the Python-based backend, where the AI-assisted parser identifies the required house parameters and converts them into structured data. The validation module then checks the extracted requirements before allowing the generation process to continue.

After successful validation, the structured house specification is passed to Blender. Blender uses the validated parameters to procedurally generate the required house structure. The generated result is then returned through the system and presented to the user through the frontend.

Testing is performed at different stages of the system to verify correct functionality, requirement interpretation, validation, communication between modules, and 3D house generation. Special attention is given to **BHK values, bedroom counts, floor counts, and floor-wise room distribution** to ensure that explicitly specified requirements are not changed during processing.

The integration and testing process includes:

1. Testing user input and validation.
2. Testing AI-assisted requirement extraction.
3. Testing structured house data generation.
4. Testing requirement and constraint validation.
5. Testing frontend-backend communication.
6. Testing Blender integration.
7. Testing multi-floor house generation.
8. Testing error handling and system feedback.
9. Testing the final 3D house output.

4.2 IMPLEMENTATION RESULTS

The implemented system successfully demonstrates the complete workflow of converting natural-language house requirements into a conceptual 3D house model. The system combines the React frontend, Python-based backend, AI-assisted requirement processing, validation, and Blender-based procedural generation into a single workflow.

The user can provide requirements such as **BHK type, number of bedrooms, number of floors, required rooms, and floor-wise room distribution** through the frontend. The system processes these requirements and converts them into a structured house specification. The validation stage checks the extracted information before the 3D generation process begins.

After successful validation, the structured requirements are passed to Blender, where the house is procedurally generated according to the available requirements. Supported architectural elements such as **floors, rooms, walls, doors, windows, stairs, and balconies** can be generated as part of the conceptual model.

The implementation provides the following results:

- Natural-language house requirements can be submitted through the web interface.
- Important house parameters can be extracted and structured.
- BHK and bedroom requirements are processed separately and consistently.
- Floor-wise room requirements can be validated and preserved.
- Invalid or conflicting requirements can be identified before generation.
- Validated requirements can be passed to the Blender generation module.
- Multi-floor conceptual house structures can be generated.

- The generated 3D house can be presented as the final output.
- Frontend and backend components can communicate as part of the complete workflow.

Overall, the implementation demonstrates the feasibility of using **AI-assisted requirement processing and procedural 3D modelling** to simplify the initial stages of house design and visualization. The system is intended as a **conceptual design and visualization tool** rather than a replacement for professional architectural or structural design

.

CHAPTER 5
CONCLUSION AND FUTURE WORKS

5.1 CONCLUSION

The **AI-Based Text-to-3D House Generation System** successfully demonstrates an approach for converting natural-language house requirements into a conceptual 3D house model. The main objective of the project is to reduce the complexity of the initial house modelling process by allowing users to describe their requirements in a simple text-based format.

The system accepts requirements such as **BHK type, number of bedrooms, number of floors, required rooms, and floor-wise room distribution**. These requirements are processed using an AI-assisted requirement parser and converted into a structured house specification. The structured information is then validated to identify inconsistencies and ensure that important constraints provided by the user are maintained.

One of the important aspects of the system is the handling of **BHK and floor-wise requirements**. For example, when a user specifies a 4BHK house with two floors and requires two bedrooms on each floor, the system should preserve the requirement as four bedrooms distributed across the two floors rather than changing the specified distribution. This improves the consistency between the user's original requirement and the generated model.

After validation, the structured requirements are passed to the Blender-based procedural generation module. Blender generates the conceptual house structure using the available parameters. Elements such as **floors, rooms, walls, doors, windows, stairs, and balconies** can be generated based on the supported requirements. This reduces the need for manually creating each basic architectural component.

The React-based frontend provides an accessible interface for entering requirements and viewing system responses, while the Python-based backend manages processing, validation, communication, and interaction with the 3D generation module. The integration of these components creates a complete workflow from **text input → requirement processing → validation → structured house data → Blender generation → 3D output**.

The project demonstrates the potential of combining **Artificial Intelligence and procedural 3D modelling** for conceptual architectural visualization. It can be particularly useful during the early design stage, where users need to visualize an idea before moving towards detailed professional architectural planning.

However, the generated output should be considered a **conceptual visualization** and not a final professional architectural or structural design. Detailed structural calculations, building-code compliance, engineering validation, construction feasibility, and professional architectural approval are outside the current scope of the system.

Overall, the project provides a foundation for an AI-assisted house design system and demonstrates how natural-language interaction can be connected with automated 3D generation to make preliminary house visualization simpler, faster, and more user-friendly.

5.2 FUTURE WORKS

Although the proposed **AI-Based Text-to-3D House Generation System** provides a functional approach for converting text-based requirements into conceptual 3D house models, several improvements can be introduced in future versions. These improvements can increase the accuracy, flexibility, visual quality, and practical usefulness of the system.

The major areas of future development include:

1. **Advanced House Layout Generation**
   Future versions can generate more detailed and optimized floor plans based on room sizes, relationships between rooms, available space, entrances, ventilation, and circulation requirements.
2. **Improved Natural Language Understanding**
   The AI requirement parser can be enhanced to understand more complex and detailed user descriptions, including multiple constraints, preferences, measurements, and architectural requirements.
3. **Advanced Floor-Wise Planning**
   The system can provide better control over the distribution of bedrooms, bathrooms, kitchens, living areas, and other rooms across multiple floors while strictly maintaining user-defined constraints.
4. **Furniture and Interior Generation**
   Future versions can automatically add furniture and interior elements such as beds, sofas, dining tables, wardrobes, kitchen units, and other commonly required components.
5. **Material and Design Customization**
   Users can be provided with options to select wall materials, floor materials, colours,

doors, windows, roofing styles, and other visual properties.

6. **Improved 3D Visualization**
   The generated models can be enhanced with better lighting, materials, textures, rendering, and interactive viewing capabilities to provide a more realistic representation of the proposed house.
7. **Multiple Design Alternatives**
   Instead of generating a single design, the system can produce multiple possible layouts based on the same requirements, allowing users to compare and select a preferred design.
8. **Architectural Constraint Support**
   Additional constraints such as plot dimensions, room dimensions, setbacks, staircase placement, parking requirements, and other design considerations can be incorporated into the generation process.
9. **Interactive Modification**
   Users could modify the generated house using additional natural-language commands such as changing the number of bedrooms, moving a room, adding a balcony, or modifying the floor arrangement without starting the generation process again.
10. **Professional Design Integration**
    In future, the generated conceptual model could be integrated with professional architectural workflows and exported into suitable formats for further editing and detailed design by architects or engineers.
11. **Virtual and Augmented Reality Support**
    The generated 3D models could be extended to VR or AR environments, allowing users to explore the proposed house from an immersive perspective.
12. **Cloud-Based Processing and Scalability**
    The system could be deployed as a cloud-based platform to support larger and more complex house-generation tasks and make the application accessible from different devices.

These future improvements can transform the current conceptual prototype into a more comprehensive **AI-assisted architectural visualization and design-support platform**. However, professional architectural and structural validation would still be required before using any generated design for actual construction.

5.2.1 Future Implementation Plan

The future implementation of the **AI-Based Text-to-3D House Generation System** can be carried out in multiple stages to improve the capabilities and practical usability of the system. The development can gradually extend the current conceptual generation workflow into a more advanced AI-assisted architectural design platform.

1 – Improved Requirement Understanding

The first stage can focus on improving the AI-assisted requirement parser. The system can be enhanced to understand more detailed natural-language descriptions, including room dimensions, preferred room locations, plot size, architectural styles, and additional user constraints.

2 – Advanced Floor Planning

The next stage can introduce improved floor-plan generation. The system can consider room dimensions, room relationships, circulation space, entrances, staircases, ventilation, and floor-wise requirements while creating the house layout.

3 – Detailed 3D Generation

The Blender generation module can be enhanced to produce more detailed architectural models. Additional elements such as furniture, roofing, interior components, improved doors and windows, materials, textures, and lighting can be incorporated.

Phase 4 – Interactive Customization

An interactive editing feature can be introduced so that users can modify the generated model through additional text commands or interface controls. For example, users could request changes such as adding a bedroom, modifying a floor, adding a balcony, or changing the room arrangement.

5 – Multiple Design Alternatives

The system can be extended to generate multiple house designs from the same requirements. Users can compare different layouts and select the design that best matches their preferences.

6 – Advanced Visualization

The generated models can be provided with improved rendering and interactive 3D viewing. Future implementations can also explore immersive technologies such as **Virtual Reality (VR)** and **Augmented Reality (AR)** for exploring the generated house.

7 – Professional Design Integration

In the final stage, the system can be integrated with professional architectural design workflows. Generated conceptual models can be exported into suitable formats for further refinement by architects and engineers. Additional building regulations and structural constraints can also be incorporated where appropriate.

Through these stages, the proposed system can evolve from a basic text-to-3D conceptual generator into a more comprehensive **AI-assisted house planning and visualization system**. The future implementation should continue to maintain validation of user requirements and clearly distinguish AI-generated conceptual designs from professionally approved architectural and structural plans.

REFERENCES

1. Blender Foundation. *Blender – Open Source 3D Creation Software*.
2. React Documentation. *React – A JavaScript Library for Building User Interfaces*.
3. Python Software Foundation. *Python Documentation*.
4. FastAPI Documentation. *FastAPI – Modern Python Web Framework for Building APIs*.
5. Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson.
6. Jurafsky, D. and Martin, J. H. *Speech and Language Processing*.
7. OpenAI. *Generative AI and Natural Language Processing Concepts*.
8. Foley, J. D., van Dam, A., Feiner, S. K. and Hughes, J. F. *Computer Graphics: Principles and Practice*. Addison-Wesley.
9. Eberly, D. H. *3D Game Engine Design: A Practical Approach to Real-Time Computer Graphics*.
10. Sommerville, I. *Software Engineering*. Pearson**.**

APPENDIXES

Appendix A- Screenshots

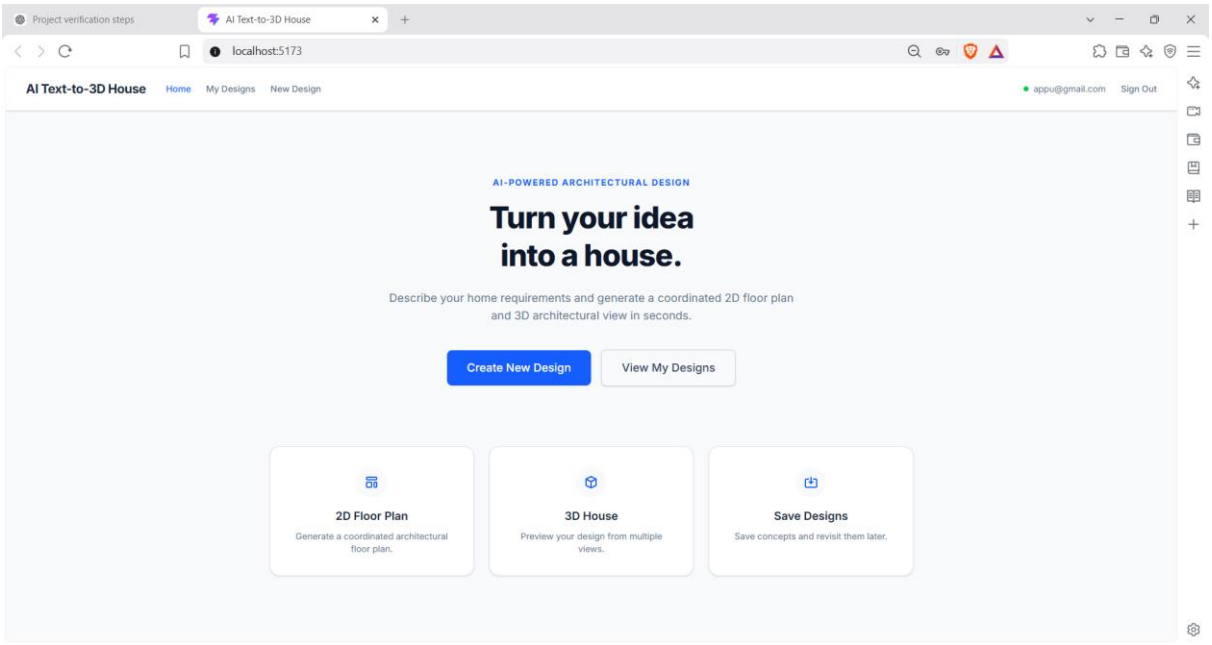


Figure A.1 Home Page

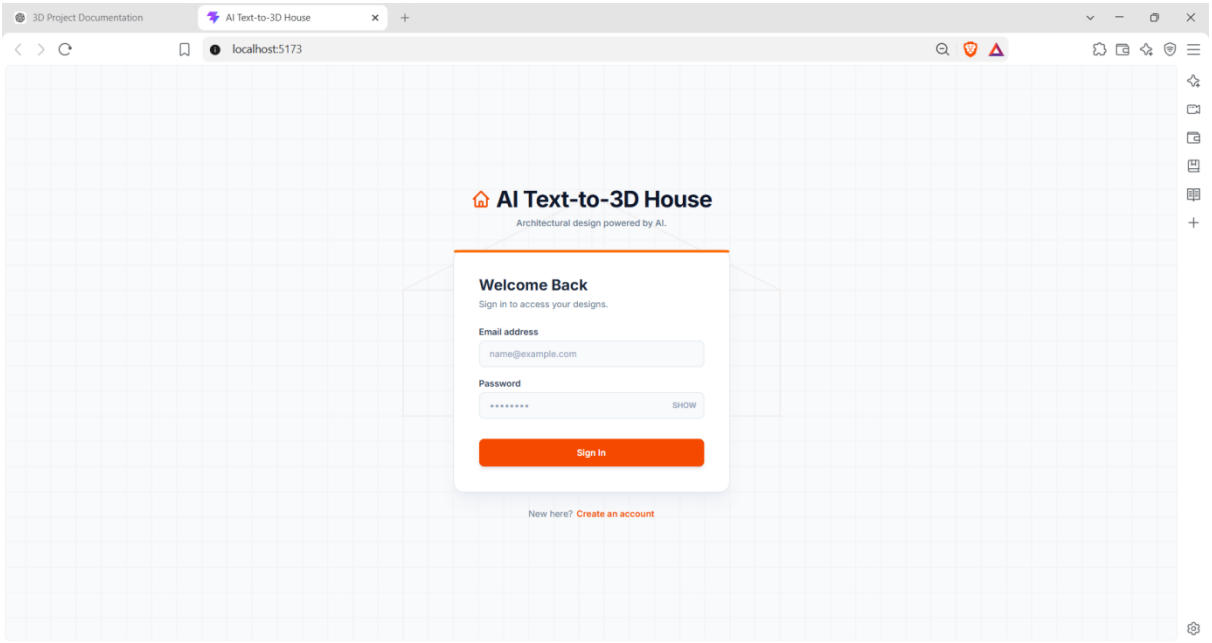


Figure A.2 Login Page

Figure A.4: Hospital Dashboard

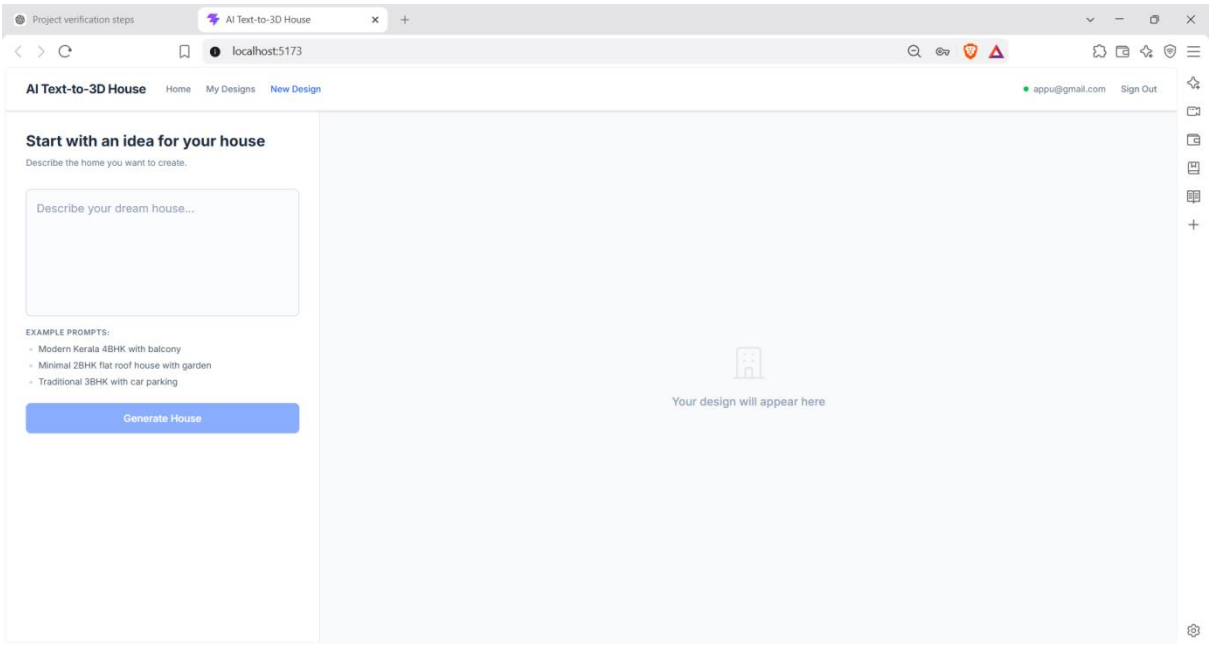


Figure A.3: AI Dashboard

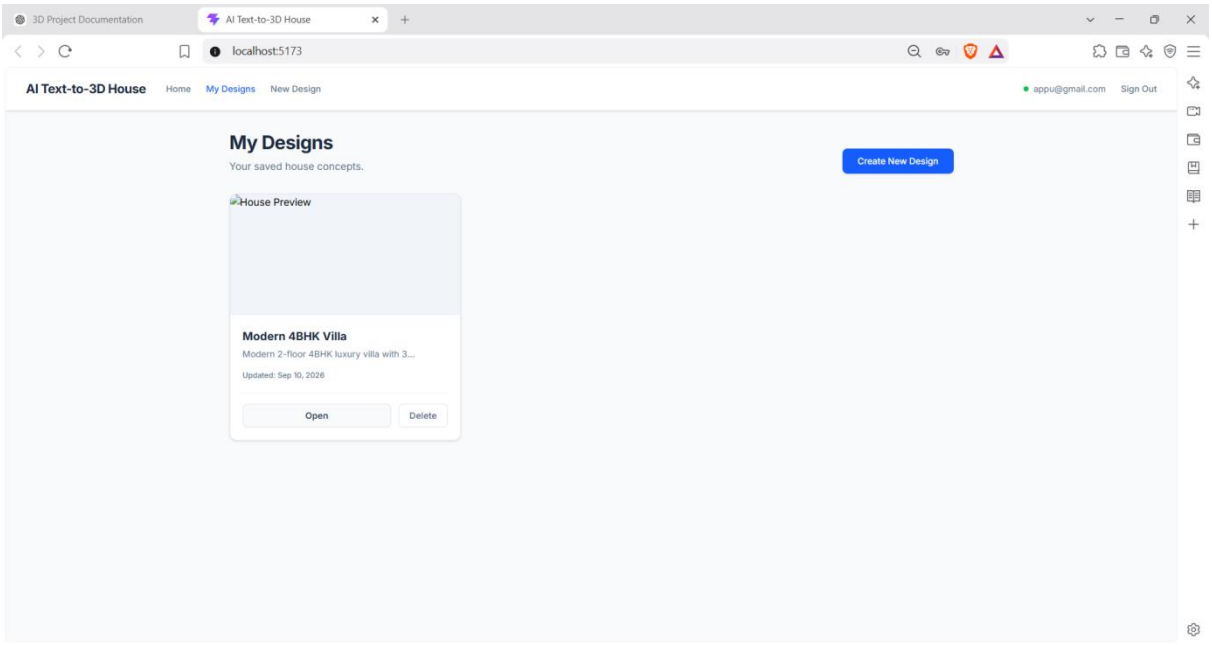


Figure A.4: Save Design Dashboard

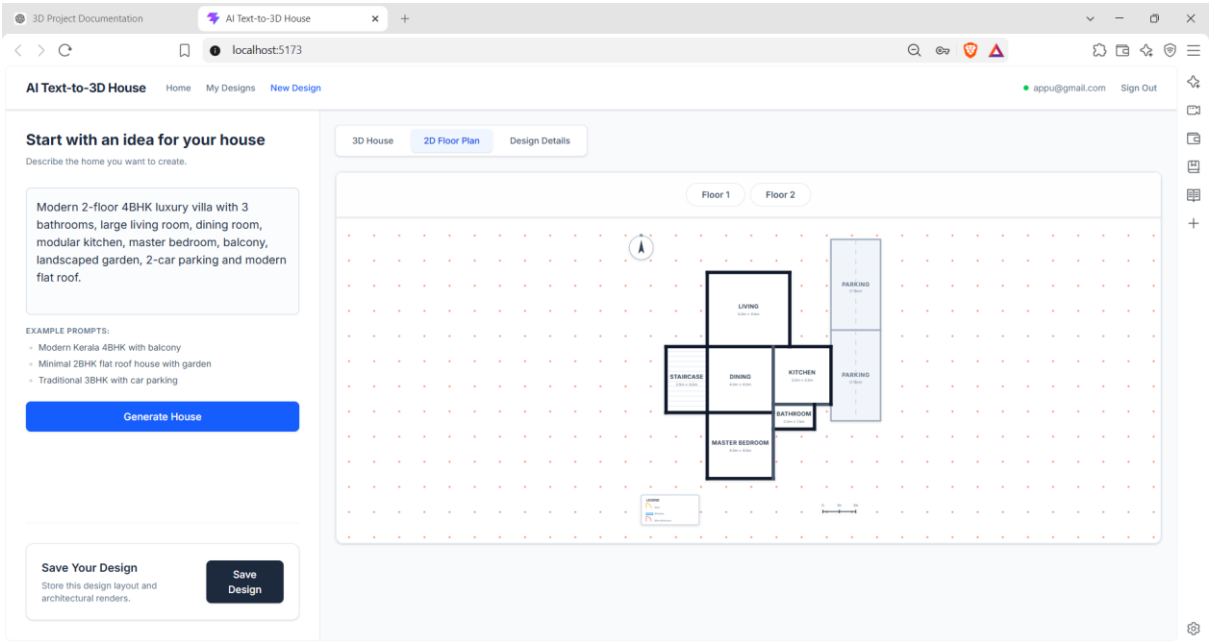


Figure A5: Final 2D House Output

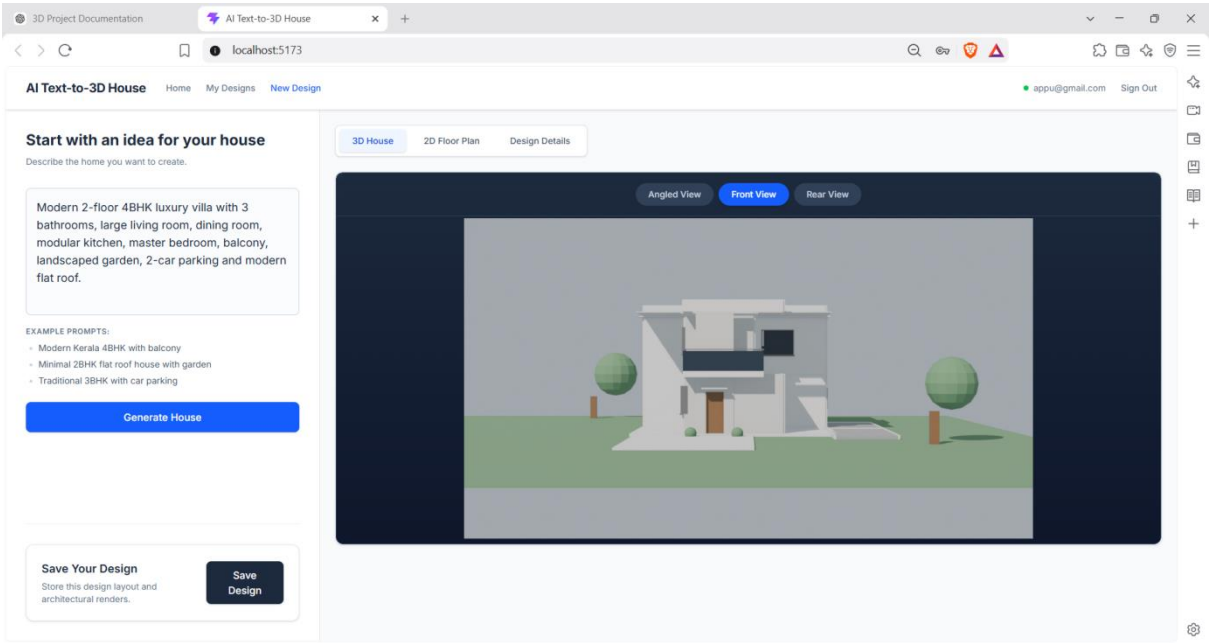


Figure A6: Final 3D House Output

Appendix B- Sample Code

B.1 Main House Generation API

```
import os
import uuid

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

# Pipeline imports
from backend.requirements.parser import parse_requirements
from backend.layout.generator import generate_layout
from backend.floorplan.generator import generate_floorplan
from backend.house3d.builder import generate_3d
from backend.house3d.blender_export import export_and_render


router = APIRouter()


class GenerateRequest(BaseModel):
    prompt: str


@router.post("/generate")
def generate_house(req: GenerateRequest):

    try:
        # Step 1: Parse natural-language house requirements
        req_output = parse_requirements(req.prompt)
```

```
# Step 2: Generate the canonical house layout
layout = generate_layout(req_output)

# Step 3: Generate the 2D floor plan
floorplan = generate_floorplan(layout)

# Step 4: Generate the 3D house scene
scene = generate_3d(floorplan)

# Step 5: Create a unique directory
# for storing generated renders
gen_id = str(uuid.uuid4())

render_dir = f"renders/{gen_id}"

os.makedirs(render_dir, exist_ok=True)

# Step 6: Export the 3D scene and generate
# different rendered views using Blender
export_and_render(scene, render_dir)

# Step 7: Prepare render paths
# for frontend display
renders = {}

front_path = os.path.join(
    render_dir,
    "front.png"
)

angled_path = os.path.join(
    render_dir,
    "angled.png"
)
```

```
rear_path = os.path.join(
    render_dir,
    "rear.png"
)

if os.path.exists(front_path):
    renders["front"] = (
        f"/renders/{gen_id}/front.png"
    )

if os.path.exists(angled_path):
    renders["angled"] = (
        f"/renders/{gen_id}/angled.png"
    )

if os.path.exists(rear_path):
    renders["rear"] = (
        f"/renders/{gen_id}/rear.png"
    )

# Step 8: Return the complete generated
# house information to the frontend
return {
    "success": True,

    "requirements":
        req_output.model_dump(),

    "layout":
        layout.model_dump(),

    "floorplan":
        floorplan.model_dump(),
```

```
            "scene":
                scene.model_dump(),

            "renders":
                renders
        }

    except Exception as e:

        import traceback

        traceback.print_exc()

        raise HTTPException(
            status_code=500,
            detail=str(e)
        )
```

B.2 Frontend–Backend API Integration Code

```
import axios from "axios";

const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL || "http://localhost:8000";

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    "Content-Type": "application/json",
  },
});
```

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("ai_house_token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});

export async function generateHouse(prompt: string) {
  const response = await api.post("/api/generate", {
    prompt,
  });

  return response.data;
}

export async function getMe() {
  const response = await api.get("/api/auth/me");
  return response.data;
}

export async function saveDesign(data: any) {
  const response = await api.post("/api/designs", data);
  return response.data;
}

export async function updateDesign(
  id: string,
  data: any
) {
  const response = await api.put(
    `/api/designs/${id}`,
```

```
    data
  );

  return response.data;
}

export async function deleteDesign(id: string) {
  const response = await api.delete(
    `/api/designs/${id}`
  );

  return response.data;
}

export async function getDesigns() {
  const response = await api.get("/api/designs");
  return response.data;
}

export default api;
```

B.3 AI-Based Requirement Parsing Code

```
import json
import re


def parse_requirements(prompt: str):
    """
    Extracts house requirements from natural-language input.
    """

    if not prompt or not prompt.strip():
        raise ValueError("House requirement cannot be empty.")
```

```
text = prompt.lower().strip()

result = {
    "house_type": None,
    "bhk": None,
    "bedrooms": 0,
    "floors": 1,
    "rooms": [],
    "floor_wise_rooms": {},
    "additional_requirements": []
}

# Detect BHK
bhk_match = re.search(r"(\d+)\s*bhk", text)

if bhk_match:
    bhk = int(bhk_match.group(1))

    result["bhk"] = f"{bhk}BHK"
    result["house_type"] = f"{bhk}BHK"
    result["bedrooms"] = bhk

# Detect number of floors
floor_match = re.search(
    r"(\d+)\s*(?:floor|floors|storey|storeys)",
    text
)

if floor_match:
    result["floors"] = int(floor_match.group(1))

# Detect common rooms
room_keywords = {
```

```
    "living room": "Living Room",
    "dining room": "Dining Room",
    "kitchen": "Kitchen",
    "bedroom": "Bedroom",
    "bathroom": "Bathroom",
    "balcony": "Balcony",
    "study room": "Study Room",
    "pooja room": "Pooja Room",
    "garage": "Garage"
}

for keyword, room_name in room_keywords.items():

    if keyword in text:

        if room_name not in result["rooms"]:
            result["rooms"].append(room_name)

# Add bedroom requirement for BHK
if result["bedrooms"] > 0:

    if "Bedroom" not in result["rooms"]:
        result["rooms"].append("Bedroom")

# Detect floor-wise bedroom distribution
ground_match = re.search(
    r"(\d+)\s*bedrooms?.*?(?:ground floor|ground)",
    text
)

first_match = re.search(
    r"(\d+)\s*bedrooms?.*?(?:first floor|first)",
    text
)
```

```
    if ground_match:
        result["floor_wise_rooms"]["Ground Floor"] = {
            "bedrooms": int(ground_match.group(1))
        }

    if first_match:
        result["floor_wise_rooms"]["First Floor"] = {
            "bedrooms": int(first_match.group(1))
        }

    # Store original requirement for traceability
    result["additional_requirements"].append(prompt)

    return result
```

B.4 Blender 3D House Generation Code

```
import bpy
import sys
import json
import os
import math


def get_args():
    try:
        idx = sys.argv.index("--")
        return sys.argv[idx + 1:]
    except ValueError:
        return []
```

```
def clean_scene():
    bpy.ops.wm.read_factory_settings(use_empty=True)


def setup_materials():
    materials = {}

    def create_mat(name, color, roughness=0.8, transmission=0.0):
        mat = bpy.data.materials.new(name=name)
        mat.use_nodes = True
        bsdf = mat.node_tree.nodes.get("Principled BSDF")

        if bsdf:
            if "Base Color" in bsdf.inputs:
                bsdf.inputs["Base Color"].default_value = color
            if "Roughness" in bsdf.inputs:
                bsdf.inputs["Roughness"].default_value = roughness
            if transmission > 0 and "Transmission Weight" in bsdf.inputs:
                bsdf.inputs["Transmission Weight"].default_value = transmission

        return mat

    materials["wall_exterior"] = create_mat(
        "wall_exterior", (0.85, 0.83, 0.80, 1.0), 0.7
    )

    materials["wall_interior"] = create_mat(
        "wall_interior", (0.7, 0.68, 0.65, 1.0), 0.8
    )

    materials["slab"] = create_mat(
        "slab", (0.8, 0.8, 0.8, 1.0), 0.8
    )
```

```
materials["roof"] = create_mat(
    "roof", (0.25, 0.25, 0.28, 1.0), 0.9
)

materials["door_wood"] = create_mat(
    "door_wood", (0.25, 0.12, 0.05, 1.0), 0.4
)

materials["door_frame"] = create_mat(
    "door_frame", (0.1, 0.1, 0.1, 1.0), 0.6
)

materials["window_glass"] = create_mat(
    "window_glass",
    (0.02, 0.03, 0.05, 1.0),
    0.05,
    0.0
)

materials["window_frame"] = create_mat(
    "window_frame", (0.15, 0.15, 0.15, 1.0), 0.5
)

materials["stairs"] = create_mat(
    "stairs", (0.6, 0.6, 0.6, 1.0), 0.7
)

materials["parking"] = create_mat(
    "parking", (0.35, 0.35, 0.35, 1.0), 0.95
)

materials["balcony"] = create_mat(
    "balcony", (0.65, 0.65, 0.65, 1.0), 0.7
```

```
    )

    materials["grass"] = create_mat(
        "grass", (0.15, 0.30, 0.15, 1.0), 0.9
    )

    return materials


def create_cube(name, x, y, z, w, d, h, mat):
    bpy.ops.mesh.primitive_cube_add(size=1)

    obj = bpy.context.active_object
    obj.name = name

    obj.location = (
        x + w / 2,
        y + d / 2,
        z + h / 2
    )

    obj.scale = (w, d, h)

    if mat:
        if obj.data.materials:
            obj.data.materials[0] = mat
        else:
            obj.data.materials.append(mat)

    return obj


def create_prism(
    name,
```

```
    x,
    y,
    z,
    w,
    d,
    h,
    mat_roof,
    mat_wall
):
    verts = [
        (x, y, z),
        (x + w, y, z),
        (x + w, y + d, z),
        (x, y + d, z),
        (x + w / 2, y, z + h),
        (x + w / 2, y + d, z + h)
    ]

    faces = [
        (0, 1, 4),
        (3, 2, 5),
        (0, 3, 5, 4),
        (1, 2, 5, 4),
        (0, 1, 2, 3)
    ]

    mesh = bpy.data.meshes.new(name + "_mesh")
    mesh.from_pydata(verts, [], faces)
    mesh.update()

    obj = bpy.data.objects.new(name, mesh)
    bpy.context.collection.objects.link(obj)

    if mat_roof:
```

```
        obj.data.materials.append(mat_roof)

    if mat_wall:
        obj.data.materials.append(mat_wall)

        if len(obj.data.polygons) > 1:
            obj.data.polygons[0].material_index = 1
            obj.data.polygons[1].material_index = 1

    return obj


def setup_lighting(lighting_data):

    bpy.ops.object.light_add(
        type="SUN",
        radius=1,
        align="WORLD",
        location=(0, 0, 0)
    )

    sun = bpy.context.active_object

    sun.data.energy = lighting_data.get(
        "sun_energy", 4.0
    )

    sun.data.use_shadow = True
    sun.data.shadow_soft_size = 0.15
    sun.data.color = (1.0, 0.95, 0.90)

    sun_pos = lighting_data.get(
        "sun_position",
        {"x": -25, "y": -45, "z": 35}
```

```
)

sun.location = (
    sun_pos["x"],
    sun_pos["y"],
    sun_pos["z"]
)

direction = -sun.location

rot_quat = direction.to_track_quat(
    "-Z",
    "Y"
)

sun.rotation_euler = rot_quat.to_euler()

bpy.ops.object.light_add(
    type="AREA",
    radius=50,
    align="WORLD",
    location=(20, 20, 20)
)

fill = bpy.context.active_object

fill.data.energy = (
    lighting_data.get("sun_energy", 3.5) * 12
)

fill.data.color = (0.75, 0.85, 0.95)

direction_fill = -fill.location
```

```
    fill.rotation_euler = (
        direction_fill
        .to_track_quat("-Z", "Y")
        .to_euler()
    )

    world = bpy.data.worlds.new("World")
    bpy.context.scene.world = world

    world.use_nodes = True

    bg_node = world.node_tree.nodes.get(
        "Background"
    )

    if bg_node:
        bg_node.inputs[0].default_value = (
            0.35,
            0.4,
            0.45,
            1.0
        )

        bg_node.inputs[1].default_value = (
            lighting_data.get(
                "ambient_energy",
                0.8
            )
        )


def render_camera(
    scene,
    output_dir,
```

```
    name,
    location,
    target
):
    cam_data = bpy.data.cameras.new(name=name)

    cam_data.lens = 60.0

    cam_obj = bpy.data.objects.new(
        name=name,
        object_data=cam_data
    )

    bpy.context.collection.objects.link(cam_obj)

    cam_obj.location = location

    import mathutils

    target_vec = mathutils.Vector(target)
    loc_vec = mathutils.Vector(location)

    direction = target_vec - loc_vec

    rot_quat = direction.to_track_quat(
        "-Z",
        "Y"
    )

    cam_obj.rotation_euler = (
        rot_quat.to_euler()
    )

    bpy.context.scene.camera = cam_obj
```

```
    bpy.context.scene.render.engine = (
        "BLENDER_EEVEE"
    )

    bpy.context.scene.render.resolution_x = 1280
    bpy.context.scene.render.resolution_y = 720

    output_path = os.path.join(
        output_dir,
        f"{name}.png"
    )

    bpy.context.scene.render.filepath = (
        output_path
    )

    bpy.ops.render.render(
        write_still=True
    )

    print(
        f"Saved render to {output_path}"
    )


def create_sphere(
    name,
    x,
    y,
    z,
    w,
    d,
    h,
```

```
    mat
):
    bpy.ops.mesh.primitive_uv_sphere_add(
        segments=16,
        ring_count=8,
        radius=1.0
    )

    obj = bpy.context.active_object
    obj.name = name

    obj.scale = (
        w / 2,
        d / 2,
        h / 2
    )

    obj.location = (
        x + w / 2,
        y + d / 2,
        z + h / 2
    )

    if mat:
        if obj.data.materials:
            obj.data.materials[0] = mat
        else:
            obj.data.materials.append(mat)

    return obj


def build_scene(scene_data):
```

```
mats = setup_materials()

for obj_data in scene_data.get(
    "objects",
    []
):

    t = obj_data["type"]

    pos = obj_data["position"]

    dim = obj_data["dimensions"]

    mat_name = obj_data["material"]

    obj_name = obj_data["id"]

    if t == "roof_pitch":

        create_prism(
            obj_name,
            pos["x"],
            pos["y"],
            pos["z"],
            dim["width"],
            dim["depth"],
            dim["height"],
            mats.get(mat_name),
            mats.get("wall_exterior")
        )

    elif t == "tree_sphere":

        create_sphere(
```

```
            obj_name,
            pos["x"],
            pos["y"],
            pos["z"],
            dim["width"],
            dim["depth"],
            dim["height"],
            mats.get(mat_name)
        )

    else:

        create_cube(
            obj_name,
            pos["x"],
            pos["y"],
            pos["z"],
            dim["width"],
            dim["depth"],
            dim["height"],
            mats.get(mat_name)
        )


def main():

    args = get_args()

    if len(args) < 2:

        print(
            "Usage: blender -b -P "
            "blender_script.py -- "
            "<scene_json> <output_dir>"
```

```
    )

    sys.exit(1)

scene_json = args[0]
output_dir = args[1]

with open(scene_json, "r") as f:

    scene_data = json.load(f)

clean_scene()

build_scene(scene_data)

setup_lighting(
    scene_data.get(
        "lighting",
        {}
    )
)

objects = [
    o for o in scene_data.get(
        "objects",
        []
    )
    if o["id"] != "landscape_grass"
]

if objects:

    min_x = min(
        o["position"]["x"]
```

```
    for o in objects
)

max_x = max(
    o["position"]["x"]
    + o["dimensions"]["width"]
    for o in objects
)

min_y = min(
    o["position"]["y"]
    for o in objects
)

max_y = max(
    o["position"]["y"]
    + o["dimensions"]["depth"]
    for o in objects
)

max_z = max(
    o["position"]["z"]
    + o["dimensions"]["height"]
    for o in objects
)

building_width = (
    max_x - min_x
)

building_depth = (
    max_y - min_y
)
```

```
center_x = (
    min_x + max_x
) / 2

center_y = (
    min_y + max_y
) / 2

center_z = max_z / 2

target_vec = (
    center_x,
    center_y,
    center_z * 0.8
)

dist_from_center_y = (
    max(
        building_width,
        max_z * 1.77
    ) * 2.5
)

dist_angled = (
    max(
        math.sqrt(
            building_width ** 2
            + building_depth ** 2
        ),
        max_z * 1.77
    ) * 2.5
)

render_camera(
```

```
    scene_data,
    output_dir,
    "front",
    (
        center_x,
        center_y - dist_from_center_y,
        max_z * 0.5
    ),
    target_vec
)

render_camera(
    scene_data,
    output_dir,
    "rear",
    (
        center_x,
        center_y + dist_from_center_y,
        max_z * 0.5
    ),
    target_vec
)

render_camera(
    scene_data,
    output_dir,
    "angled",
    (
        center_x - dist_angled * 0.7,
        center_y - dist_angled * 0.7,
        max_z * 0.8
    ),
    target_vec
)
```

```
if __name__ == "__main__":
    main()
```

B.5 House Requirement Validation Code

```
def validate_requirements(requirements):
    """
    Validate the extracted house requirements
    before starting 3D generation.
    """

    errors = []

    # Validate BHK
    bhk = requirements.get("bhk")

    if bhk:
        try:
            bhk_number = int(
                str(bhk).replace("BHK", "").strip()
            )

            if bhk_number <= 0:
                errors.append(
                    "BHK value must be greater than zero."
                )

        except ValueError:
            errors.append(
                "Invalid BHK format."
            )
```

```
# Validate bedroom count
bedrooms = requirements.get(
    "bedrooms",
    0
)

if bedrooms < 0:
    errors.append(
        "Bedroom count cannot be negative."
    )

# Validate floor count
floors = requirements.get(
    "floors",
    1
)

if floors <= 0:
    errors.append(
        "Number of floors must be greater than zero."
    )

# Validate floor-wise requirements
floor_wise_rooms = requirements.get(
    "floor_wise_rooms",
    {}
)

floor_bedrooms = 0

for floor, room_data in floor_wise_rooms.items():

    if not isinstance(room_data, dict):
        errors.append(
```

```
            f"Invalid room data for {floor}."
        )
        continue

    floor_bedrooms += room_data.get(
        "bedrooms",
        0
    )

# Ensure floor-wise bedroom count
# does not exceed total requirement
if floor_bedrooms > bedrooms:

    errors.append(
        "Floor-wise bedroom distribution "
        "exceeds the total bedroom requirement."
    )

# Validate floor references
for floor in floor_wise_rooms.keys():

    floor_name = floor.lower()

    if (
        "ground" in floor_name
        and floors < 1
    ):
        errors.append(
            "Ground floor is not valid."
        )

    if (
        "first" in floor_name
        and floors < 2
```

```
        ):
            errors.append(
                "First floor requires at least "
                "two floors."
            )

    # Final validation result
    if errors:

        return {
            "valid": False,
            "errors": errors
        }

    return {
        "valid": True,
        "errors": []
    }
```

B.6 Frontend 3D House Viewer Code

```
import { useEffect, useRef } from "react";
import * as THREE from "three";
import { OrbitControls } from "three/examples/jsm/controls/OrbitControls";

interface House3DViewerProps {
  modelUrl?: string;
}

export function House3DViewer({
  modelUrl,
}: House3DViewerProps) {
  const containerRef = useRef<HTMLDivElement>(null);
```

```
useEffect(() => {
  if (!containerRef.current) return;

  const container = containerRef.current;

  const scene = new THREE.Scene();

  scene.background = new THREE.Color(0xf1f5f9);

  const camera = new THREE.PerspectiveCamera(
    45,
    container.clientWidth /
      container.clientHeight,
    0.1,
    1000
  );

  camera.position.set(8, 6, 8);

  const renderer =
    new THREE.WebGLRenderer({
      antialias: true,
    });

  renderer.setSize(
    container.clientWidth,
    container.clientHeight
  );

  renderer.setPixelRatio(
    window.devicePixelRatio
  );

  container.appendChild(
```

```
  renderer.domElement
);

// Lighting
const ambientLight =
  new THREE.AmbientLight(
    0xffffff,
    1.5
  );

scene.add(ambientLight);

const directionalLight =
  new THREE.DirectionalLight(
    0xffffff,
    2
  );

directionalLight.position.set(
  10,
  15,
  10
);

scene.add(
  directionalLight
);

// Ground
const groundGeometry =
  new THREE.PlaneGeometry(
    30,
    30
  );
```

```
const groundMaterial =
  new THREE.MeshStandardMaterial({
    color: 0xd9dee5,
  });

const ground =
  new THREE.Mesh(
    groundGeometry,
    groundMaterial
  );

ground.rotation.x =
  -Math.PI / 2;

scene.add(ground);

// Example house object
const houseGroup =
  new THREE.Group();

const wallGeometry =
  new THREE.BoxGeometry(
    5,
    3,
    5
  );

const wallMaterial =
  new THREE.MeshStandardMaterial({
    color: 0xe5e7eb,
  });

const house =
```

```
  new THREE.Mesh(
    wallGeometry,
    wallMaterial
  );

house.position.y = 1.5;

houseGroup.add(house);

// Roof
const roofGeometry =
  new THREE.ConeGeometry(
    3.8,
    2,
    4
  );

const roofMaterial =
  new THREE.MeshStandardMaterial({
    color: 0x475569,
  });

const roof =
  new THREE.Mesh(
    roofGeometry,
    roofMaterial
  );

roof.position.y = 4;

roof.rotation.y =
  Math.PI / 4;

houseGroup.add(roof);
```

```
scene.add(
  houseGroup
);

// Camera controls
const controls =
  new OrbitControls(
    camera,
    renderer.domElement
  );

controls.enableDamping = true;

controls.target.set(
  0,
  2,
  0
);

// Animation
const animate = () => {
  requestAnimationFrame(
    animate
  );

  controls.update();

  renderer.render(
    scene,
    camera
  );
};
```

```
animate();

// Responsive viewer
const handleResize = () => {
  if (!container) return;

  camera.aspect =
    container.clientWidth /
    container.clientHeight;

  camera.updateProjectionMatrix();

  renderer.setSize(
    container.clientWidth,
    container.clientHeight
  );
};

window.addEventListener(
  "resize",
  handleResize
);

return () => {
  window.removeEventListener(
    "resize",
    handleResize
  );

  controls.dispose();

  renderer.dispose();

  if (
```

```
        container.contains(
          renderer.domElement
        )
      ) {
        container.removeChild(
          renderer.domElement
        );
      }
    };
  }, [modelUrl]);

  return (
    <div
      ref={containerRef}
      className="w-full h-full min-h-[500px]"
    />
  );
}
```